CS 240 — Data Structures and Data Management

Vishesh Gupta

University of Waterloo

Contents

1 Introduction and Asymptotic Analysis **1**
- 1.1 Problems, Algorithms and Programs . . . 1
- 1.2 The RAM Model . . . 1
- 1.3 Asymptotic Notation . . . 2
  - 1.3.1 The Limit Rule . . . 2
  - 1.3.2 Useful Properties . . . 3
  - 1.3.3 Common Growth Rates . . . 3
- 1.4 Worst-, Best- and Average-Case . . . 3
- 1.5 Analysing Pseudocode . . . 3
  - 1.5.1 Useful Sums . . . 4
- 1.6 MergeSort and Recurrences . . . 4
  - 1.6.1 Common Recurrences . . . 5
- 1.7 Summary . . . 5

2 Priority Queues **6**
- 2.1 Abstract Data Types . . . 6
- 2.2 The Priority Queue ADT . . . 6
  - 2.2.1 PQ-Sort . . . 6
  - 2.2.2 Simple Implementations . . . 6
- 2.3 Binary Heaps . . . 7
  - 2.3.1 Array Storage . . . 7
  - 2.3.2 Insertion: fix-up (bubble-up) . . . 7
  - 2.3.3 deleteMax: fix-down (bubble-down) . . . 8
- 2.4 Building a Heap: Heapify . . . 8
- 2.5 HeapSort . . . 9
- 2.6 Other Heap Uses . . . 9
- 2.7 Summary . . . 10

3 Sorting and Randomized Algorithms **11**
- 3.1 The Selection Problem . . . 11
  - 3.1.1 Partition . . . 11
  - 3.1.2 QuickSelect . . . 11
- 3.2 Randomized Algorithms . . . 12
- 3.3 QuickSort . . . 13
  - 3.3.1 Average-Case Analysis . . . 13
  - 3.3.2 Practical Improvements . . . 13
- 3.4 Lower Bound for Comparison Sorting . . . 14

3.5 Non-Comparison Sorting . . . 14
3.5.1 Counting Sort (Key-Indexed Sort) . . . 14
3.5.2 LSD Radix Sort . . . 14
3.5.3 MSD Radix Sort . . . 15
3.6 Summary of Sorting Algorithms . . . 15

4 Dictionaries and Balanced Search Trees 16
4.1 The Dictionary ADT . . . 16
4.2 Binary Search Trees . . . 16
4.3 AVL Trees . . . 17
4.4 Rotations . . . 17
4.5 AVL Insertion . . . 18
4.6 AVL Deletion . . . 18
4.7 Summary . . . 19

5 Other Dictionary Implementations 20
5.1 Skip Lists . . . 20
5.1.1 Search . . . 20
5.1.2 Insert . . . 21
5.1.3 Delete . . . 21
5.1.4 Analysis . . . 21
5.2 Re-ordering Items . . . 22
5.2.1 Optimal Static Ordering . . . 22
5.2.2 Dynamic Ordering: Move-To-Front and Transpose . . . 22
5.3 Summary . . . 23

6 Dictionaries for Special Keys 24
6.1 Lower Bound for Search . . . 24
6.2 Interpolation Search . . . 24
6.3 Tries . . . 25
6.3.1 Operations . . . 25
6.4 Variations of Tries . . . 25
6.4.1 No leaf labels . . . 25
6.4.2 Pruned tries . . . 26
6.4.3 Compressed Tries (Patricia Tries) . . . 26
6.4.4 Multiway Tries . . . 26
6.5 Summary . . . 27

7 Dictionaries via Hashing 28
7.1 Direct Addressing . . . 28
7.2 Hashing . . . 28
7.3 Separate Chaining . . . 28
7.4 Open Addressing . . . 29
7.4.1 Linear Probing . . . 29
7.4.2 Quadratic Probing . . . 29
7.4.3 Double Hashing . . . 29
7.4.4 Cuckoo Hashing . . . 30
7.4.5 Expected Probes . . . 30

7.5 Choosing a Hash Function . . . 30
7.5.1 Universal Hashing . . . 31
7.6 Hashing vs. Balanced Search Trees . . . 31
7.7 Summary . . . 31

8 Range Searching and Multidimensional Data 32
8.1 Range Queries . . . 32
8.1.1 Range Search in a BST . . . 32
8.2 Multidimensional Data . . . 33
8.3 Quadtrees . . . 33
8.4 kd-Trees . . . 33
8.5 Range Trees . . . 34
8.6 Comparison . . . 35
8.7 Summary . . . 35

9 String Matching 36
9.1 The Problem . . . 36
9.2 Brute Force . . . 36
9.3 Knuth–Morris–Pratt (KMP) . . . 36
9.4 Boyer–Moore . . . 37
9.5 Rabin–Karp Fingerprinting . . . 38
9.6 Suffix Tries and Suffix Trees . . . 39
9.7 Suffix Arrays . . . 39
9.8 Summary . . . 40

10 Compression 41
10.1 Encoding Basics . . . 41
10.1.1 Character Encodings . . . 41
10.2 Huffman Coding . . . 41
10.3 Run-Length Encoding . . . 42
10.4 Lempel–Ziv–Welch (LZW) . . . 43
10.5 Move-To-Front Transform . . . 44
10.6 Burrows–Wheeler Transform . . . 44
10.7 Summary . . . 45

11 External Memory 46
11.1 The External Memory Model . . . 46
11.2 External Sorting . . . 46
11.3 External Dictionaries: B-Trees . . . 47
11.3.1 Multiway Search Trees . . . 47
11.3.2 2-3 Trees . . . 47
11.3.3 (a,b)-Trees . . . 48
11.3.4 B-Trees . . . 48
11.4 Extendible Hashing . . . 49
11.5 Summary . . . 50

Chapter 1

Introduction and Asymptotic Analysis

CS 240 is about storing and querying data efficiently. Almost every topic in the course follows the same pattern: define an *abstract data type* (ADT) by the operations it supports, give several *implementations*, and compare them by the running time and space of each operation. This first module sets up the language we use to make such comparisons.

1.1 Problems, Algorithms and Programs

Definition 1.1. A *problem* is a description of the relationship between inputs and outputs. A *problem instance* is one particular input. The *size* of an instance, written n, is a positive integer measuring how big the input is (usually the number of items).

Definition 1.2. An *algorithm* is a finite, step-by-step procedure that solves every instance of a problem. A *program* is an implementation of an algorithm in a particular programming language.

Given a problem we want to (1) design an algorithm, (2) argue it is correct, and (3) analyse its efficiency, before ever writing a program. We compare algorithms by their resource usage as a function of n: *running time* and *auxiliary space*.

1.2 The RAM Model

Timing a program on a real machine depends on the hardware, compiler, and load. Instead we count operations in an idealised machine.

Definition 1.3 (Random Access Machine)**.** The RAM model has an unbounded set of memory cells, each holding one "word" (a number or pointer of reasonable size). Any cell can be read or written in constant time, and each *primitive operation* (arithmetic, comparison, assignment, following a pointer, array indexing, function call/return) takes constant time.

The running time of an algorithm on an input I is then the number of primitive operations it performs, $T(I)$. The space is the maximum number of memory cells used beyond the input.

Remark. The RAM model assumes words have $\Theta(\log n)$ bits; it does not let us pack an unbounded amount of information into one word. This matters for the lower bounds later in the course and is why radix sort is not "cheating".

1.3 Asymptotic Notation

We care about growth rate for large n and ignore constant factors (which depend on the machine anyway).

Definition 1.4 (Order notation)**.** Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$.

- $f(n) \in \mathcal{O}(g(n))$ if there exist $c > 0$ and $n_0 > 0$ such that $f(n) \leq c\,g(n)$ for all $n \geq n_0$. (Asymptotic upper bound.)
- $f(n) \in \Omega(g(n))$ if there exist $c > 0$, $n_0 > 0$ with $f(n) \geq c\,g(n)$ for all $n \geq n_0$. (Asymptotic lower bound.)
- $f(n) \in \Theta(g(n))$ if there exist $c_1, c_2 > 0$, n_0 with $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$. (Tight bound.)
- $f(n) \in o(g(n))$ if for *every* $c > 0$ there is n_0 with $f(n) < c\,g(n)$ for all $n \geq n_0$. (Strictly smaller growth.)
- $f(n) \in \omega(g(n))$ if for every $c > 0$ there is n_0 with $f(n) > c\,g(n)$ for all $n \geq n_0$. (Strictly larger growth.)

Note the quantifier difference: for $\mathcal{O}$ we get to *choose* one c; for o the inequality must hold for *all* c.

Example 1.5. Show $2n^2 + 3n + 11 \in \mathcal{O}(n^2)$. For $n \geq 1$ we have $3n \leq 3n^2$ and $11 \leq 11n^2$, so $2n^2 + 3n + 11 \leq 16n^2$. Take $c = 16$, $n_0 = 1$.

Example 1.6. Show $n^2 \notin \mathcal{O}(n)$. Suppose $n^2 \leq cn$ for all $n \geq n_0$. Then $n \leq c$ for all $n \geq n_0$, which fails at $n = \max(n_0, \lceil c \rceil + 1)$.

1.3.1 The Limit Rule

Proving bounds from the definitions is tedious; the limit rule is usually faster.

Theorem 1.7 (Limit rule)**.** *Suppose $f(n), g(n) > 0$ for large n and $L = \lim_{n\to\infty} f(n)/g(n)$ exists (possibly ∞). Then*

$$f(n) \in \begin{cases} o(g(n)) & L = 0, \\ \Theta(g(n)) & 0 < L < \infty, \\ \omega(g(n)) & L = \infty. \end{cases}$$

The converse does not hold: the limit might not exist (e.g. $f(n) = n(2 + \sin n)$ vs. $g(n) = n$, which are Θ of each other).

Example 1.8. $\log n \in o(n^\epsilon)$ for any $\epsilon > 0$: by L'Hôpital, $\lim \frac{\ln n}{n^\epsilon} = \lim \frac{1/n}{\epsilon n^{\epsilon-1}} = \lim \frac{1}{\epsilon n^\epsilon} = 0$. Similarly $n^k \in o(b^n)$ for any constant k and $b > 1$.

1.3.2 Useful Properties

- Identity / transitivity: $f \in \Theta(f)$; if $f \in \mathcal{O}(g)$ and $g \in \mathcal{O}(h)$ then $f \in \mathcal{O}(h)$ (same for the others).
- $f \in \Theta(g) \iff f \in \mathcal{O}(g)$ and $f \in \Omega(g)$.
- $f \in \mathcal{O}(g) \iff g \in \Omega(f)$; $f \in o(g) \iff g \in \omega(f)$.
- Sums: $f + g \in \Theta(\max\{f, g\})$.
- Constants and logs: $\log_a n \in \Theta(\log_b n)$ for any bases $a, b > 1$, so we write just $\log n$.

1.3.3 Common Growth Rates

Name	Function	Doubling n multiplies T by
constant	$\Theta(1)$	1
logarithmic	$\Theta(\log n)$	adds a constant
linear	$\Theta(n)$	2
linearithmic	$\Theta(n \log n)$	slightly more than 2
quadratic	$\Theta(n^2)$	4
cubic	$\Theta(n^3)$	8
exponential	$\Theta(2^n)$	squares it

Ordering: $1 \ll \log n \ll \sqrt{n} \ll n \ll n \log n \ll n^2 \ll n^3 \ll 2^n \ll n! \ll n^n$, where $\ll$ means "is o of".

1.4 Worst-, Best- and Average-Case

The running time depends on the instance, not just n. Define

$$T^{\text{worst}}(n) = \max\{T(I) : \text{size}(I) = n\},$$
$$T^{\text{best}}(n) = \min\{T(I) : \text{size}(I) = n\},$$
$$T^{\text{avg}}(n) = \frac{1}{|\mathcal{I}_n|} \sum_{\text{size}(I)=n} T(I).$$

Unless stated otherwise, "the running time" means the worst case. Note that $\mathcal{O}$ and worst case are independent ideas: we can give $\mathcal{O}$, Ω or Θ bounds on any of the three functions.

Common pitfall

Saying "algorithm A is $\mathcal{O}(n^2)$ and B is $\mathcal{O}(n \log n)$, so B is faster" is not justified: $\mathcal{O}$ is only an upper bound, and A might in fact be $\Theta(n)$. To compare algorithms we need Θ bounds. Also, asymptotic bounds say nothing about small n.

1.5 Analysing Pseudocode

The strategy: assign $\Theta(1)$ to each primitive line, multiply by the number of loop iterations, and add. Work from the inside out. It is fine to derive a Θ bound directly, or to find matching $\mathcal{O}$ and Ω bounds separately (often easier).

```
Test1(n):
  sum <- 0
  for i <- 1 to n do
    for j <- i to n do
      sum <- sum + (i - j)^2
  return sum
```

The inner body is $\Theta(1)$ and runs $n - i + 1$ times, so the total is $\sum_{i=1}^{n}(n - i + 1) = \sum_{k=1}^{n} k = \frac{n(n+1)}{2} \in \Theta(n^2)$.

```
Test2(A, n):
  max <- 0
  for i <- 1 to n do
    for j <- i to n do
      sum <- 0
      for k <- i to j do
        sum <- sum + A[k]
      max <- maximum(max, sum)
  return max
```

The count is $\sum_{i=1}^{n}\sum_{j=i}^{n}(j - i + 1)$. For an upper bound each term is at most n, giving $\mathcal{O}(n^3)$. For a lower bound, restrict to $i \le n/4$ and $j \ge 3n/4$: there are at least $(n/4)^2$ such pairs, each with $j - i + 1 \ge n/2$, giving at least $n^3/32 \in \Omega(n^3)$. Hence $\Theta(n^3)$.

> The "restrict the range" trick is the standard way to get lower bounds on nested sums without computing them exactly: throw away terms to get a smaller sum that is easy to bound below.

1.5.1 Useful Sums

- Arithmetic: $\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \in \Theta(n^2)$; more generally $\sum_{i=1}^{n} i^k \in \Theta(n^{k+1})$.
- Geometric: $\sum_{i=0}^{n-1} r^i = \frac{r^n - 1}{r - 1}$; this is $\Theta(r^n)$ if $r > 1$, $\Theta(n)$ if $r = 1$, $\Theta(1)$ if $0 < r < 1$.
- Harmonic: $H_n = \sum_{i=1}^{n} \frac{1}{i} = \ln n + \gamma + o(1) \in \Theta(\log n)$.
- $\sum_{i=1}^{n} \log i = \log n! \in \Theta(n \log n)$.
- $\sum_{i \ge 0} i/2^i = 2$ (used for heapify).

1.6 MergeSort and Recurrences

```
MergeSort(A, l, r):        // sorts A[l..r]
  if r <= l then return
  m <- floor((l + r) / 2)
  MergeSort(A, l, m)
  MergeSort(A, m+1, r)
  Merge(A, l, m, r)

Merge(A, l, m, r):         // A[l..m], A[m+1..r] sorted
  copy A[l..r] into S[l..r]
  i <- l; j <- m+1
```

```
for k <- l to r do
  if i > m then A[k] <- S[j]; j <- j+1
  else if j > r then A[k] <- S[i]; i <- i+1
  else if S[i] <= S[j] then A[k] <- S[i]; i <- i+1
  else A[k] <- S[j]; j <- j+1
```

Merge takes $\Theta(r - l + 1)$ time and uses $\Theta(n)$ auxiliary space for S. The running time satisfies

$$T(n) = \begin{cases} T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n) & n > 1, \\ \Theta(1) & n = 1. \end{cases}$$

It is standard to drop floors/ceilings and solve the "sloppy" recurrence $T(n) = 2T(n/2) + cn$ for n a power of 2.

Solving by unrolling.

$$\begin{aligned} T(n) &= 2T(n/2) + cn = 4T(n/4) + 2cn = 8T(n/8) + 3cn = \cdots \\ &= 2^k T(n/2^k) + kcn. \end{aligned}$$

With $k = \log_2 n$: $T(n) = n\,T(1) + cn \log n \in \Theta(n \log n)$.

Recursion-tree view. Level i of the tree has 2^i subproblems of size $n/2^i$, so each level costs cn; there are $\log n + 1$ levels.

1.6.1 Common Recurrences

Recurrence	Solution	Example
$T(n) = T(n/2) + \Theta(1)$	$\Theta(\log n)$	binary search
$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$	mergesort
$T(n) = 2T(n/2) + \Theta(\log n)$	$\Theta(n)$	heapify
$T(n) = T(cn) + \Theta(n)$, $0 < c < 1$	$\Theta(n)$	selection (expected)
$T(n) = 2T(n/4) + \Theta(1)$	$\Theta(\sqrt{n})$	kd-tree range search
$T(n) = T(\sqrt{n}) + \Theta(1)$	$\Theta(\log \log n)$	interpolation search (avg)
$T(n) = T(n-1) + \Theta(n)$	$\Theta(n^2)$	quicksort worst case
$T(n) = 2T(n-1) + \Theta(1)$	$\Theta(2^n)$	naive recursion

The Master Theorem (from CS 341) covers $T(n) = aT(n/b) + \Theta(n^d)$: compare d with $\log_b a$. In CS 240 we mostly just unroll or quote the table.

1.7 Summary

- Algorithms are analysed in the RAM model by counting primitive operations.
- $\mathcal{O}, \Omega, \Theta$ give upper, lower, tight bounds up to constants; o, ω are their strict versions. The limit rule is the fastest tool.
- Worst/best/average case is a separate choice from which bound we give.
- For nested loops, sum from the inside out; get lower bounds by discarding terms.
- MergeSort runs in $\Theta(n \log n)$ via $T(n) = 2T(n/2) + \Theta(n)$ and uses $\Theta(n)$ extra space.

Chapter 2

Priority Queues

2.1 Abstract Data Types

Definition 2.1. An *abstract data type* (ADT) describes a collection of data and the operations on it, without saying how it is stored. A *data structure* is a concrete implementation of an ADT.

Familiar examples: a *stack* (LIFO; `push`, `pop`) and a *queue* (FIFO; `enqueue`, `dequeue`), both implementable with arrays or linked lists so that every operation is $\Theta(1)$ (amortized for a dynamic array).

2.2 The Priority Queue ADT

Definition 2.2. A *priority queue* stores items, each with a *priority* (key), and supports

- `insert(x)`: add item x with its priority;
- `deleteMax()`: remove and return an item of highest priority.

(A min-oriented priority queue uses `deleteMin` instead.) Optional extras: `size`, `isEmpty`, `getMax`.

Stacks and queues are special cases (priority = insertion time, or its negative).

2.2.1 PQ-Sort

Any priority queue gives a sorting algorithm:

```
PQ-Sort(A[0..n-1]):
  PQ <- empty priority queue
  for k <- 0 to n-1 do PQ.insert(A[k])
  for k <- n-1 down to 0 do A[k] <- PQ.deleteMax()
```

The running time is $\mathcal{O}(n \cdot \text{insert} + n \cdot \text{deleteMax})$.

2.2.2 Simple Implementations

Implementation	`insert`	`deleteMax`	PQ-Sort is…
Unsorted array/list	$\Theta(1)$	$\Theta(n)$	selection sort, $\Theta(n^2)$
Sorted array/list	$\Theta(n)$	$\Theta(1)$	insertion sort, $\Theta(n^2)$
Binary heap	$\Theta(\log n)$	$\Theta(\log n)$	heapsort, $\Theta(n \log n)$

2.3 Binary Heaps

Definition 2.3. A (max-)*heap* is a binary tree satisfying

1. **Structural property:** every level is full except possibly the last, which is filled from the left.
2. **Heap-order property:** for every node v other than the root, $\text{key}(\text{parent}(v)) \geq \text{key}(v)$.

Consequently the maximum is at the root. The heap is *not* sorted: siblings are in no particular order.

Lemma 2.4. *A heap with n nodes has height $\Theta(\log n)$.*

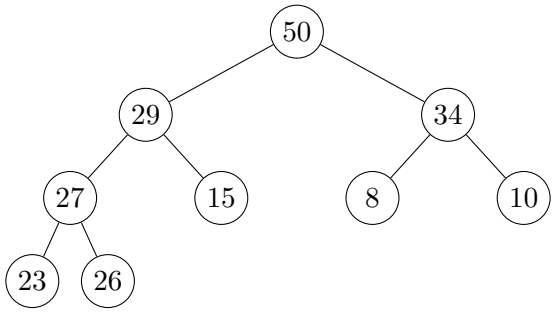
Proof. If the height is h, levels $0, \ldots, h-1$ are full, so $n \geq 2^h$; and the tree has at most $2^{h+1} - 1$ nodes. Thus $2^h \leq n \leq 2^{h+1} - 1$, so $h = \lfloor \log_2 n \rfloor$. □

2.3.1 Array Storage

Because of the structural property, store the nodes level by level, left to right, in an array $A[0..n-1]$. No pointers are needed:

- root at $A[0]$;
- children of $A[i]$ at $A[2i+1]$ and $A[2i+2]$;
- parent of $A[i]$ $(i > 0)$ at $A[\lfloor (i-1)/2 \rfloor]$;
- the last node is $A[n-1]$.

Example 2.5. The array $[50, 29, 34, 27, 15, 8, 10, 23, 26]$ is a heap: e.g. $A[1] = 29$ has children $A[3] = 27$ and $A[4] = 15$; $A[3] = 27$ has children 23 and 26.



2.3.2 Insertion: fix-up (bubble-up)

Place the new item in the next free spot (preserving structure), then swap it upward while it is larger than its parent.

```
insert(x):
  A[n] <- x; n <- n + 1
  fix-up(A, n-1)

fix-up(A, k):
  while k > 0 and A[parent(k)] < A[k] do
    swap A[k] and A[parent(k)]
    k <- parent(k)
```

Each swap moves up a level, so the time is $\mathcal{O}(\text{height}) = \mathcal{O}(\log n)$.

2.3.3 deleteMax: fix-down (bubble-down)

Remove the root, move the last item to the root, then swap it down with its *larger* child while it is smaller than that child.

```
deleteMax():
  max <- A[0]
  swap A[0] and A[n-1]; n <- n - 1
  fix-down(A, n, 0)
  return max

fix-down(A, n, k):
  while k is not a leaf do       // 2k+1 < n
    j <- 2k+1                    // left child
    if j+1 < n and A[j+1] > A[j] then j <- j+1
    if A[k] >= A[j] then break
    swap A[k] and A[j]
    k <- j
```

Again $\mathcal{O}(\log n)$.

Pitfall

In fix-down you must swap with the *larger* child; swapping with the smaller one breaks heap order between the two children.

Example 2.6. `deleteMax` on the heap above: return 50, move 26 to the root, giving

$$[26, 29, 34, 27, 15, 8, 10, 23].$$

The larger child of 26 is 34: swap, giving

$$[34, 29, 26, 27, 15, 8, 10, 23].$$

Now 26 at index 2 has children 8, 10: stop.

2.4 Building a Heap: Heapify

Given n items in an array, we want to make it a heap.

Option 1 (top-down): insert items one at a time with fix-up. Worst case $\Theta(n \log n)$ (e.g. sorted increasing input: every item bubbles to the root).

Option 2 (bottom-up, Floyd): treat the array as a tree and call fix-down on each internal node, from the last internal node back to the root.

```
heapify(A, n):
  for i <- parent(n-1) down to 0 do
    fix-down(A, n, i)
```

When we fix-down at node i, both subtrees of i are already heaps, so afterward the subtree at i is a heap (by induction).

Theorem 2.7. *Bottom-up heapify runs in $\Theta(n)$ time.*

Proof sketch. Let $h = \lfloor \log n \rfloor$. A node at height j (distance j from the bottom) costs $\mathcal{O}(j)$ in fix-down, and there are at most $\lceil n/2^{j+1} \rceil$ nodes at height j. So the total is

$$\sum_{j=0}^{h} \frac{n}{2^{j+1}} \cdot \mathcal{O}(j) = \mathcal{O}\left(n \sum_{j \geq 0} \frac{j}{2^j} \right) = \mathcal{O}(2n) = \mathcal{O}(n).$$

Clearly $\Omega(n)$ since every item is touched. □

Intuition: most nodes are near the bottom, and those nodes only travel a short distance. In top-down insertion most nodes are near the bottom and travel a *long* distance.

Example 2.8. Heapify $[3, 1, 6, 5, 2, 4]$. Internal nodes are indices $2, 1, 0$.

- $i = 2$ (6): child 4; $6 \geq 4$, no change.
- $i = 1$ (1): children 5, 2; swap with 5: $[3, 5, 6, 1, 2, 4]$.
- $i = 0$ (3): children 5, 6; swap with 6: $[6, 5, 3, 1, 2, 4]$; then 3 at index 2 has child 4; swap: $[6, 5, 4, 1, 2, 3]$.

2.5 HeapSort

```
HeapSort(A, n):
  heapify(A, n)                 // Theta(n)
  while n > 1 do
    swap A[0] and A[n-1]        // move max to its final position
    n <- n - 1
    fix-down(A, n, 0)
```

- Time: $\Theta(n) + n \cdot \mathcal{O}(\log n) = \mathcal{O}(n \log n)$, and it is $\Theta(n \log n)$ in the worst case.
- Space: *in place*, $\Theta(1)$ auxiliary space (the heap shrinks as the sorted suffix grows).
- Not stable.

2.6 Other Heap Uses

Finding the k-th largest. Heapify then k deleteMax calls: $\mathcal{O}(n + k \log n)$. Alternatively keep a min-heap of size k while scanning: $\mathcal{O}(n \log k)$. (Chapter 3 does this in $\Theta(n)$ expected time.)

Merging k sorted lists. Keep the heads of the lists in a min-heap of size k; total $\mathcal{O}(n \log k)$.

Other PQ realisations exist (meldable heaps, binomial heaps) that also support efficient `merge`; a randomized meldable heap merges two heaps in expected $\mathcal{O}(\log n)$ by recursively merging into a randomly chosen subtree. Some terms covered these briefly as enrichment.

2.7 Summary

- Priority queue ADT: `insert`, `deleteMax`. Any PQ gives a sort.
- A binary heap is a nearly complete binary tree with heap order, stored in an array with index arithmetic.
- `insert` (fix-up) and `deleteMax` (fix-down) take $\mathcal{O}(\log n)$.
- Bottom-up heapify is $\Theta(n)$ because $\sum j/2^j$ converges.
- HeapSort: $\Theta(n \log n)$ worst case, in place, not stable.

Chapter 3

Sorting and Randomized Algorithms

3.1 The Selection Problem

Definition 3.1. *Selection*: given an array A of n numbers and $0 \le k < n$, find the item that would be at index k if A were sorted. Special cases: $k = 0$ (minimum), $k = n - 1$ (maximum), $k = \lfloor n/2 \rfloor$ (median).

Easy solutions: sort ($\Theta(n \log n)$), or use a heap ($\mathcal{O}(n + k \log n)$). We want $\Theta(n)$.

3.1.1 Partition

Both QuickSelect and QuickSort rely on two subroutines:

- `choose-pivot(A)`: return an index p (simplest: the last index).
- `partition(A, p)`: rearrange A so that all items $\le A[p]$ come first, then the pivot at index i, then all items $> A[p]$; return i.

An efficient in-place partition (Hoare-style, as in the course):

```
partition(A, p):
  swap A[n-1] and A[p]          // pivot now at the end
  v <- A[n-1]
  i <- -1; j <- n-1
  loop
    do i <- i+1 while i < n and A[i] < v
    do j <- j-1 while j > 0 and A[j] > v
    if i >= j then break
    swap A[i] and A[j]
  swap A[n-1] and A[i]          // put pivot in place
  return i
```

Invariant: $A[0..i-1] \le v$ and $A[j+1..n-2] \ge v$. Runs in $\Theta(n)$ time with $\mathcal{O}(1)$ extra space.

3.1.2 QuickSelect

```
quick-select(A, k):
  p <- choose-pivot(A)
  i <- partition(A, p)
  if i = k then return A[i]
```

```
  else if i > k then return quick-select(A[0..i-1], k)
  else return quick-select(A[i+1..n-1], k - i - 1)
```

Worst case. If the pivot is always the largest item and $k = 0$, we recurse on size $n - 1$ every time: $T(n) = T(n-1) + cn$, so $\Theta(n^2)$.

Best case. The first pivot is the answer: $\Theta(n)$.

Average case. Assume all $n!$ input permutations are equally likely (equivalently, the pivot's final position i is uniform on $\{0, \dots, n-1\}$ and the subarrays are again random). Then

$$T^{\text{avg}}(n) \le cn + \frac{1}{n} \sum_{i=0}^{n-1} \max\{T(i), T(n-i-1)\}.$$

Theorem 3.2. *$T^{avg}(n) \in \Theta(n)$ for QuickSelect.*

Proof sketch. Call the pivot *good* if $n/4 \le i < 3n/4$; this happens with probability $1/2$. After a good pivot the recursion has size at most $3n/4$; after a bad one, at most n. Then $T(n) \le cn + \frac{1}{2}T(3n/4) + \frac{1}{2}T(n)$, so $T(n) \le 2cn + T(3n/4)$, which unrolls to $T(n) \le 2cn \sum_{j \ge 0} (3/4)^j = 8cn$. □

3.2 Randomized Algorithms

The average-case analysis assumes the input is random, which we cannot control (an adversary could feed sorted data). Instead we make the *algorithm* random.

Definition 3.3. A *randomized algorithm* uses random bits. For a fixed instance I and random outcome R, the running time is $T(I, R)$. The *expected running time* is $T^{\exp}(I) = \mathbb{E}_R[T(I,R)] = \sum_R T(I,R) \Pr[R]$, and the *worst-case expected time* is $T^{\exp}(n) = \max_{|I|=n} T^{\exp}(I)$.

Two approaches:

1. Shuffle the input randomly first (in $\Theta(n)$ by Fisher–Yates), then run the deterministic algorithm.
2. Choose the pivot uniformly at random: `p <- random(n)`.

With a random pivot, *every* input has the same expected behaviour as the average-case analysis, so randomized QuickSelect has expected time $\Theta(n)$ on every instance. The worst case is still $\Theta(n^2)$ but occurs with tiny probability and not for any particular input.

```
shuffle(A):
  for i <- n-1 down to 1 do
    swap A[i] and A[random(i+1)]    // random(m) uniform in {0..m-1}
```

> There is also a deterministic $\Theta(n)$ worst-case selection algorithm (median-of-medians, choosing a pivot from medians of groups of 5). It is mostly of theoretical interest because of its large constant; CS 341 covers it.

3.3 QuickSort

```
quick-sort(A):
  if n <= 1 then return
  p <- choose-pivot(A)
  i <- partition(A, p)
  quick-sort(A[0..i-1])
  quick-sort(A[i+1..n-1])
```

- Worst case: $T(n) = T(n-1) + \Theta(n) \in \Theta(n^2)$ (e.g. sorted input with last-element pivot).
- Best case: pivot always the median: $T(n) = 2T(n/2) + \Theta(n) \in \Theta(n \log n)$.

3.3.1 Average-Case Analysis

Let $T(n)$ be the average number of comparisons. Each pivot position i is equally likely, so

$$T(n) = (n-1) + \frac{1}{n}\sum_{i=0}^{n-1}\left(T(i) + T(n-1-i)\right) = (n-1) + \frac{2}{n}\sum_{i=0}^{n-1} T(i).$$

Theorem 3.4. *The average (and randomized expected) running time of QuickSort is $\Theta(n \log n)$; precisely, the expected number of comparisons is $2(n+1)H_n - 4n \approx 1.39\, n \log_2 n$.*

Proof sketch 1: the recurrence. Multiply by n: $nT(n) = n(n-1) + 2\sum_{i<n} T(i)$. Subtract the same identity for $n-1$: $nT(n) - (n-1)T(n-1) = 2(n-1) + 2T(n-1)$, so $nT(n) = (n+1)T(n-1) + 2(n-1)$. Divide by $n(n+1)$:

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2(n-1)}{n(n+1)} \le \frac{T(n-1)}{n} + \frac{2}{n+1}.$$

Telescoping gives $T(n)/(n+1) \le 2H_{n+1}$, so $T(n) \in \mathcal{O}(n \log n)$. □

Proof sketch 2: indicator variables. Let $z_1 < \cdots < z_n$ be the items in sorted order. Two items are compared at most once, and only if one of them is the pivot. Items z_i and z_j $(i < j)$ are compared iff the first pivot chosen from $\{z_i, \ldots, z_j\}$ is z_i or z_j, which has probability $\frac{2}{j-i+1}$. By linearity of expectation,

$$\mathbb{E}[\#\text{comparisons}] = \sum_{i<j} \frac{2}{j-i+1} \le \sum_{i=1}^{n}\sum_{d=2}^{n} \frac{2}{d} \le 2nH_n \in \mathcal{O}(n \log n).$$

□

3.3.2 Practical Improvements

- Random pivot, or median-of-three, to avoid sorted-input worst case.
- Recurse on the *smaller* side first and loop on the larger side, so the stack depth is $\mathcal{O}(\log n)$ worst case.
- Stop recursing at small sizes (e.g. $n \le 10$) and finish with one pass of insertion sort.
- Handle many duplicates with three-way partition ($<$, $=$, $>$).

3.4 Lower Bound for Comparison Sorting

Definition 3.5. A *comparison-based* sorting algorithm accesses keys only by comparing pairs (`A[i] < A[j]?`).

Theorem 3.6. *Every comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case (and also on average).*

Proof. Model any such algorithm on inputs of size n as a *decision tree*: each internal node is a comparison, each branch an outcome, each leaf an output permutation. Each of the $n!$ input orderings of distinct keys must reach a different leaf (otherwise two different inputs get the same rearrangement, and one of them is sorted wrongly). A binary tree with at least $n!$ leaves has height at least $\log_2(n!)$. Finally

$$\log(n!) = \sum_{i=1}^{n} \log i \geq \sum_{i=n/2}^{n} \log \frac{n}{2} \geq \frac{n}{2} \log \frac{n}{2} \in \Omega(n \log n).$$

The worst-case number of comparisons is the height, so it is $\Omega(n \log n)$. □

So MergeSort and HeapSort are asymptotically optimal among comparison sorts. To beat $n \log n$ we must use more than comparisons, e.g. the digits of the keys.

3.5 Non-Comparison Sorting

Assume keys are integers in $\{0, \ldots, R-1\}$ (or strings of m digits in radix R).

3.5.1 Counting Sort (Key-Indexed Sort)

```
counting-sort(A, R):           // keys in 0..R-1
  C[0..R-1] <- all 0
  for i <- 0 to n-1 do C[A[i]] <- C[A[i]] + 1
  I[0] <- 0                    // I[k] = first index for key k
  for k <- 1 to R-1 do I[k] <- I[k-1] + C[k-1]
  B <- copy of A
  for i <- 0 to n-1 do
    A[I[B[i]]] <- B[i]
    I[B[i]] <- I[B[i]] + 1
```

Time $\Theta(n + R)$, space $\Theta(n + R)$. It is *stable*: equal keys keep their relative order, because we scan B left to right.

3.5.2 LSD Radix Sort

Keys have m digits in base R. Sort by the least significant digit first, using a stable sort (counting sort) at each step.

```
LSD-radix-sort(A):
  for d <- m down to 1 do      // digit m is least significant
    counting-sort A by digit d
```

Correctness (by induction): after processing digits $d..m$, the array is sorted by the last $m - d + 1$ digits; stability ensures ties on digit d keep the order given by the less significant digits. Time $\Theta(m(n + R))$, space $\Theta(n + R)$.

Example 3.7. Sort $\{123, 230, 21, 320, 210, 232, 101\}$ ($R = 10$, $m = 3$, pad 21 as 021):

- by last digit: 230, 320, 210, 021, 101, 232, 123;
- by middle digit: 101, 210, 320, 021, 123, 230, 232;
- by first digit: 021, 101, 123, 210, 230, 232, 320.

3.5.3 MSD Radix Sort

Sort by the most significant digit, then recursively sort each group by the remaining digits.

```
MSD-radix-sort(A, l, r, d):
  if l < r then
    counting-sort A[l..r] by digit d
    if d < m then
      for each digit value k with a nonempty group A[l_k..r_k] do
        MSD-radix-sort(A, l_k, r_k, d+1)
```

Time $\mathcal{O}(mnR)$ in the worst case due to many tiny subproblems that each pay $\Theta(R)$; in practice one switches to another sort for small groups. MSD can stop early on distinct prefixes, and handles variable-length strings naturally.

3.6 Summary of Sorting Algorithms

Algorithm	Worst	Average/Expected	Space	Stable?
Selection sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	no
Insertion sort	$\Theta(n^2)$	$\Theta(n^2)$ (best $\Theta(n)$)	$\Theta(1)$	yes
MergeSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	yes
HeapSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(1)$	no
QuickSort (random)	$\Theta(n^2)$	$\Theta(n \log n)$ expected	$\Theta(\log n)$	no
Counting sort	$\Theta(n + R)$	$\Theta(n + R)$	$\Theta(n + R)$	yes
LSD radix sort	$\Theta(m(n + R))$	$\Theta(m(n + R))$	$\Theta(n + R)$	yes

- QuickSelect solves selection in $\Theta(n)$ expected (randomized) time.
- Randomization moves the average case from “random input” to “random choices”, giving guarantees for every input.
- Comparison sorts need $\Omega(n \log n)$ by the decision-tree argument; radix sorts sidestep this by looking at digits.

Chapter 4

Dictionaries and Balanced Search Trees

4.1 The Dictionary ADT

Definition 4.1. A *dictionary* stores a collection of *key-value pairs* (KVPs), with distinct, comparable keys, and supports

- `search(k)` (also called `find` or `lookup`);
- `insert(k, v)`;
- `delete(k)`.

We usually assume keys can be compared in $\Theta(1)$ time and that each KVP takes $\Theta(1)$ space. Elementary implementations:

Implementation	`search`	`insert`	`delete`
Unsorted array / list	$\Theta(n)$	$\Theta(1)$	$\Theta(n)$
Sorted array	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$
BST (worst)	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
AVL tree	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$

(For the unsorted list, `insert` is $\Theta(1)$ only if we do not check for duplicates; `delete` needs a search first.)

4.2 Binary Search Trees

Definition 4.2. A *binary search tree* is a binary tree in which every node stores a KVP and, for every node with key k, all keys in its left subtree are less than k and all keys in its right subtree are greater than k.

An in-order traversal of a BST lists the keys in sorted order.

```
BST-search(T, k):
  if T is empty then return "not found"
  if k = T.key then return T
```

```
  if k < T.key then return BST-search(T.left, k)
  else return BST-search(T.right, k)

BST-insert(T, k, v):
  search for k; it ends at an empty subtree
  replace that empty subtree by a new leaf (k, v)
```

Deletion. Find the node x with key k.

- x is a leaf: remove it.
- x has one child: replace x by that child.
- x has two children: find its *in-order successor* y (leftmost node in x's right subtree) or predecessor; copy y's KVP into x and delete y instead, which has at most one child.

All three operations take $\Theta(h)$ where h is the height. The height can be as bad as $n-1$ (insert keys in sorted order) and is at least $\lceil \log(n+1) \rceil - 1$. For random insertion order the expected height is $\Theta(\log n)$, but we want a worst-case guarantee.

4.3 AVL Trees

Definition 4.3. Define the height of an empty tree as -1. The *balance* of a node is height(right) − height(left). An *AVL tree* is a BST in which every node has balance in $\{-1, 0, +1\}$.

Each node stores its height (or just its balance) so it can be checked in $\Theta(1)$.

Theorem 4.4. *An AVL tree with n nodes has height $\Theta(\log n)$.*

Proof. Let $N(h)$ be the minimum number of nodes in an AVL tree of height h. Then $N(-1) = 0$, $N(0) = 1$, and a minimal tree of height h has a root with subtrees of heights $h-1$ and $h-2$, so

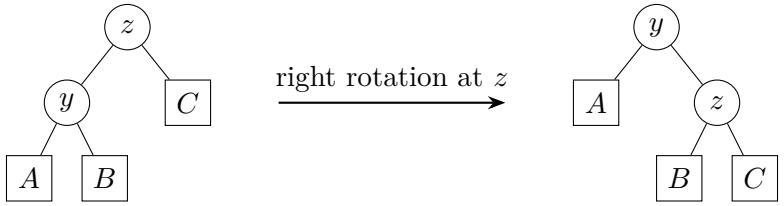
$$N(h) = N(h-1) + N(h-2) + 1.$$

This is Fibonacci-like: $N(h) = F_{h+3} - 1$, which grows like φ^h where $\varphi = (1+\sqrt{5})/2 \approx 1.618$. More simply, $N(h) > 2N(h-2)$, so $N(h) \geq 2^{h/2}$, giving $h \leq 2\log_2 n$. Hence $h \in \mathcal{O}(\log n)$; and any binary tree has $h \in \Omega(\log n)$. (The precise bound is $h < 1.44 \log_2(n+2)$.) □

So `search` is $\Theta(\log n)$. The work is in keeping the tree AVL after insertion and deletion.

4.4 Rotations

A rotation restructures a subtree in $\Theta(1)$ time while preserving the BST order property.



```
rotate-right(z):
  y <- z.left
  z.left <- y.right            // subtree B moves
  y.right <- z
  update heights of z, then y
  return y                     // new root of this subtree
```

The left rotation is symmetric. In-order order $A < y < B < z < C$ is unchanged.

4.5 AVL Insertion

Insert as in a BST; then walk back up toward the root, updating heights. At the first node z whose balance becomes ± 2, fix it with a rotation; afterward the subtree has its original height, so no ancestors need fixing (at most one *restructure* per insertion).

Let y be the taller child of z and x the taller child of y (the side of the new node). There are four cases:

Case	Shape	Fix
left-left	$y = z$.left, $x = y$.left	single right rotation at z
right-right	$y = z$.right, $x = y$.right	single left rotation at z
left-right	$y = z$.left, $x = y$.right	left rotation at y, then right at z
right-left	$y = z$.right, $x = y$.left	right rotation at y, then left at z

In every case the middle key of $\{x, y, z\}$ becomes the root of the subtree, with the other two as its children.

```
AVL-insert(T, k, v):
  z <- BST-insert(T, k, v)     // z = new leaf
  z.height <- 0
  while z is not the root do
    z <- parent(z)
    if |z.left.height - z.right.height| = 2 then
      y <- taller child of z
      x <- taller child of y   // ties: prefer the single-rotation case
      z <- restructure(x, y, z)
      break                    // for insert, one fix suffices
    update z.height
```

Example 4.5. Insert 10, 20, 30 into an empty AVL tree. After 30, node 10 has balance +2 (right-right): left rotation at 10 gives root 20 with children 10 and 30. Now insert 25 then 27: 25 goes under 30's left; 27 goes under 25's right. Node 30 has balance -2 with $y = 25$ (left) and $x = 27$ (right of 25): a left-right case. Rotate left at 25, then right at 30: 27 becomes the subtree root with children 25 and 30.

4.6 AVL Deletion

Delete as in a BST (removing a node with at most one child). Walk up from the parent of the removed node, updating heights, and restructure every unbalanced node. Here y is the taller child of z; if y's children have equal height, choose x on the same side as y so a single rotation is used.

Unlike insertion, a restructure may *decrease* the subtree's height, so imbalance can propagate: up to $\Theta(\log n)$ rotations, but still $\Theta(\log n)$ total.

4.7 Summary

Operation	BST worst	AVL worst	Rotations (AVL)
`search`	$\Theta(n)$	$\Theta(\log n)$	0
`insert`	$\Theta(n)$	$\Theta(\log n)$	at most 1 restructure
`delete`	$\Theta(n)$	$\Theta(\log n)$	up to $\Theta(\log n)$

- The dictionary ADT is the central ADT of the course.
- BSTs are simple but their height depends on insertion order.
- AVL trees keep balances in $\{-1, 0, 1\}$, which forces height $\leq 1.44 \log n$ (via the Fibonacci-like recurrence).
- Rotations fix imbalances in constant time while preserving BST order.

Chapter 5

Other Dictionary Implementations

AVL trees give $\Theta(\log n)$ worst case, but they are fiddly to implement and use extra space for heights. This module looks at two alternatives: a randomized structure (skip lists) that is easy to code with the same expected bounds, and the idea of reordering items when the access pattern is not uniform.

5.1 Skip Lists

Definition 5.1. A *skip list* is a hierarchy of sorted linked lists $S_0, S_1, \ldots, S_h$:

- each list starts with a sentinel $-\infty$ and ends with $+\infty$;
- S_0 contains every key;
- each S_{i+1} is a subset of S_i (a key in S_i is copied into S_{i+1} with some probability, independently);
- S_h contains only the sentinels.

Each node (a *tower* element) has pointers `after` (next in the same list) and `below` (same key in the list beneath).

Example 5.2.
```
S3: -inf ---------------------------------------- +inf
S2: -inf ------------- 37 ----------------------- +inf
S1: -inf ----- 23 ---- 37 ------------ 83 ------- +inf
S0: -inf - 10 - 23 - 31 - 37 - 44 - 65 - 83 - 92 - +inf
```

5.1.1 Search

Start at the top-left sentinel. Move right while the next key is smaller than k; otherwise drop down. Record the path (the last node visited on each level) because insert and delete need it.

```
get-predecessors(k):
  p <- topmost left sentinel
  P <- empty stack
  P.push(p)
  while p.below != nil do
    p <- p.below
    while p.after.key < k do p <- p.after
    P.push(p)
```

```
  return P                      // P.top() is the predecessor of k in S0

skip-search(L, k):
  P <- get-predecessors(k)
  p0 <- P.top()
  if p0.after.key = k then return p0.after
  else return "not found"
```

5.1.2 Insert

Flip a fair coin until tails; if we got i heads, the new key goes into lists $S_0, \ldots, S_i$. If $i \geq h$, first add new empty levels so that the top list stays sentinel-only. Then splice the key in after each predecessor in the stack P, from bottom up, linking `below` pointers.

```
skip-insert(L, k, v):
  i <- 0
  while random(2) = 1 do i <- i + 1     // tower height
  while height(L) <= i do add a new empty top level
  P <- get-predecessors(k)
  below <- nil
  for level <- 0 to i do
    p <- P.pop()                         // predecessor on this level
    z <- new node (k, v) with z.below <- below
    z.after <- p.after; p.after <- z
    below <- z
```

5.1.3 Delete

Find the predecessors, then on each level where `p.after.key = k`, unlink it. Afterwards remove extra empty top lists so only one sentinel-only list remains.

5.1.4 Analysis

Let tower heights be geometric: $\Pr[\text{height} \geq i] = 2^{-i}$.

- **Space.** Expected list sizes: $\mathbb{E}|S_i| = n/2^i$, so the expected total number of nodes is $\sum_i n/2^i = 2n \in \mathcal{O}(n)$.
- **Height.** $\Pr[\text{some tower has height} \geq i] \leq n/2^i$. With $i = 3 \log n$ this is $\leq 1/n^2$, so the height is $\mathcal{O}(\log n)$ with high probability; expected height $\leq \log n + 2$.
- **Search time.** Count drop-downs plus forward steps. Drop-downs are the height. Analyse the search path backward: from a node at level ℓ, we came from the left (a scan step) if the node's tower stops at ℓ, or from above if it continues — each with probability $1/2$. So the expected number of scan steps per level is at most 1, giving expected search time $\mathcal{O}(\log n)$.

Operation	Expected	Worst case
`search`, `insert`, `delete`	$\mathcal{O}(\log n)$	unbounded / $\Theta(n)$ per level
space	$\mathcal{O}(n)$	unbounded

Skip lists are simple, need no rotations, and are used in practice (e.g. in concurrent data structures). Their bounds are *expected*: the randomness is in the coin flips, not in the input.

5.2 Re-ordering Items

So far every key was assumed equally likely to be accessed. In practice accesses are skewed (the 80/20 rule): a few items are requested most of the time. If we store items in an unsorted list/array, placing popular items near the front pays off.

5.2.1 Optimal Static Ordering

Suppose we know the access probabilities $p_1, \dots, p_n$ of the items and do not change the list. Accessing the item at position i (1-indexed) costs i comparisons. The expected search cost of an ordering is

$$C = \sum_{i=1}^{n} i \cdot p_{(i)},$$

where $p_{(i)}$ is the probability of the item at position i.

Theorem 5.3. *The expected cost is minimised by sorting items in non-increasing order of access probability.*

Proof. Exchange argument: if positions $i < j$ hold items with $p_{(i)} < p_{(j)}$, swapping them changes the cost by $(ip_{(j)} + jp_{(i)}) - (ip_{(i)} + jp_{(j)}) = (j - i)(p_{(i)} - p_{(j)}) < 0$. So any optimal ordering has no such inversions. □

Example 5.4. Keys A, B, C, D, E with probabilities $\frac{2}{20}, \frac{8}{20}, \frac{1}{20}, \frac{4}{20}, \frac{5}{20}$. In the given order, the expected cost is $\frac{1\cdot2+2\cdot8+3\cdot1+4\cdot4+5\cdot5}{20} = \frac{62}{20} = 3.1$. In the optimal order B, E, D, A, C it is $\frac{8+10+12+8+5}{20} = \frac{43}{20} = 2.15$.

5.2.2 Dynamic Ordering: Move-To-Front and Transpose

When probabilities are unknown or change over time, use a self-adjusting heuristic.

- **Move-To-Front (MTF):** after a successful search, move the found item to the front of the list. Insert new items at the front.
- **Transpose:** after a successful search, swap the found item with the item immediately before it.

```
MTF-search(L, k):
  find k at position i by linear scan     // cost i
  if found then remove it and insert it at the front
```

MTF works well for a linked list (moving is $\Theta(1)$ once found); in an array it costs $\Theta(i)$ to shift, which is the same order as the search, so it is fine. Transpose is natural for arrays.

Comparison.

- MTF adapts quickly to changes in the access pattern (temporal locality), but one access to a rare item moves it to the front.
- Transpose changes slowly and is more stable, but has bad worst cases: two items at the back that are accessed alternately just swap with each other forever, each access costing $\Theta(n)$.

- MTF is *2-competitive*: its total cost on any access sequence is at most twice that of the optimal static ordering (for that sequence) plus lower-order terms. Transpose has no such guarantee.

Example 5.5. List $[A, B, C, D, E]$, accesses D, D, B, D.

- MTF: D (cost 4) gives $[D, A, B, C, E]$; D (1); B (3) gives $[B, D, A, C, E]$; D (2). Total 10.
- Transpose: D (4) gives $[A, B, D, C, E]$; D (3) gives $[A, D, B, C, E]$; B (3) gives $[A, B, D, C, E]$; D (3). Total 13.

The same MTF idea reappears in Chapter 10 as a compression transform: frequently recurring symbols get small indices.

5.3 Summary

- Skip lists: towers of geometric height; expected $\mathcal{O}(n)$ space and $\mathcal{O}(\log n)$ search/insert/delete; simple to implement.
- With known access probabilities, the best static unsorted order is by decreasing probability (exchange argument).
- With unknown probabilities, self-adjusting lists: MTF (2-competitive, adapts fast) or Transpose (conservative, bad worst case).

Chapter 6

Dictionaries for Special Keys

The comparison lower bound applies to searching too: any comparison-based search in a set of n keys needs $\Omega(\log n)$ comparisons (a decision tree must have $n+1$ distinguishable outcomes). To do better we must exploit the structure of the keys: numbers (interpolation search) or strings (tries).

6.1 Lower Bound for Search

Theorem 6.1. *In the comparison model,* `search` *requires* $\Omega(\log n)$ *comparisons in the worst case.*

Proof. A decision tree for search must have at least $n+1$ leaves (one for each key found, plus "not found"). A binary tree with $n+1$ leaves has height at least $\log(n+1)$. □

6.2 Interpolation Search

Binary search on a sorted array always probes the middle. If keys are numbers spread roughly uniformly, we can guess where k should be, like looking up a name in a phone book.

```
interpolation-search(A[0..n-1], k):
  l <- 0; r <- n - 1
  while l <= r and A[l] <= k <= A[r] do
    if A[l] = A[r] then
      if A[l] = k then return l else return "not found"
    m <- l + floor( (k - A[l]) / (A[r] - A[l]) * (r - l) )
    if A[m] = k then return m
    else if A[m] < k then l <- m + 1
    else r <- m - 1
  return "not found"
```

Binary search uses $m = l + \left\lfloor \frac{1}{2}(r-l) \right\rfloor$; interpolation search replaces $\frac{1}{2}$ by the fraction $\frac{k-A[l]}{A[r]-A[l]}$.

Example 6.2. $A = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90]$, search 70: $m = 0 + \left\lfloor \frac{70}{90} \cdot 9 \right\rfloor = 7$. Found in one probe.

Analysis.

- If keys are uniformly distributed, the expected time satisfies $T(n) = T(\sqrt{n}) + \Theta(1)$, giving $\Theta(\log \log n)$. (Intuition: after one probe the expected distance to the target is about $\sqrt{n}$.)

- Worst case $\Theta(n)$: e.g. $A = [1, 2, \ldots, n-1, 10^{100}]$, search $n-1$: each probe moves l by one.

A variant that combines interpolation with a fallback to binary steps (or galloping) avoids the linear worst case while keeping good average behaviour.

6.3 Tries

Now keys are strings over an alphabet Σ (often $\Sigma = \{0, 1\}$). Comparing two strings costs time proportional to their common prefix, so a balanced BST of strings of length m costs $\mathcal{O}(m \log n)$ per search. Tries get $\mathcal{O}(m)$.

Definition 6.3. A *trie* (from re*trie*val; also *radix tree* or *digital search tree*) is a tree in which each edge is labelled with a character, and the path from the root to a node spells a prefix. Each key is stored at the node reached by its characters.

Remark. To ensure no key is a prefix of another (which would put a key at an internal node), append an end-of-word symbol \$ to every key. Then all keys are stored at leaves. Alternatively, mark internal nodes with a flag "a word ends here".

Example 6.4. Binary trie for $\{00\$, 0001\$, 01001\$, 011\$, 0110\$, 110\$, 1101\$, 111\$\}$: from the root, branch on bit 0, bit 1, ..., with each \$ edge ending at a leaf.

6.3.1 Operations

```
trie-search(v, x, d):     // v node, x key, d = current depth
  if v is a leaf then return v
  c <- x[d]  (or $ if d = |x|)
  if v has a child w on edge c then
    return trie-search(w, x, d+1)
  else return "not found"
```

- `search(x)`: follow the characters of x; fail if a child is missing.
- `insert(x)`: search; when a child is missing, create new nodes for the remaining characters (ending with a leaf for \$).
- `delete(x)`: search to the leaf, delete it, then delete ancestors that now have no children.

All run in $\Theta(|x|)$ time, independent of n. Other useful operations: prefix search (all keys starting with p), successor/predecessor, and sorting (a pre-order traversal outputs keys in lexicographic order).

6.4 Variations of Tries

6.4.1 No leaf labels

Keys need not be stored at leaves, since the path spells them; saves space.

6.4.2 Pruned tries

Stop adding nodes once the key is unique: a node has a child only if it has at least two descendants. The leaf stores the full key, and search must compare x with the stored key at the end. This is the trie implicit in MSD radix sort.

6.4.3 Compressed Tries (Patricia Tries)

Definition 6.5. A *compressed trie* (Patricia: Practical Algorithm To Retrieve Information Coded In Alphanumeric) removes all internal nodes with only one child. Each internal node stores the *index* of the character to test next; each leaf stores the full key.

Since every internal node has at least two children, a compressed trie with n keys has at most $n-1$ internal nodes: $\Theta(n)$ space regardless of key lengths.

```
patricia-search(v, x):
  if v is a leaf then
    if x = v.key then return v else return "not found"
  d <- v.index                  // character position to test
  c <- x[d]  (or $ if d = |x|; "not found" if d > |x|)
  if v has a child w on edge c then return patricia-search(w, x)
  else return "not found"
```

Note the final comparison with the stored key is required, since skipped positions were never checked.

Insert. Search for x; this leads to some leaf (or fails at a node). Compare x with any key y in that subtree to find the first index d where they differ. Walk back up to where d belongs (between a parent with smaller index and a child with larger index), and create a new internal node with index d there, with two children: the old subtree and a new leaf for x.

Delete. Remove the leaf; if its parent now has only one child, remove the parent and connect the sibling to the grandparent.

All operations take $\mathcal{O}(|x|)$ time.

6.4.4 Multiway Tries

For a larger alphabet (e.g. $|\Sigma| = 26$ letters plus \$), each node has up to $|\Sigma|+1$ children. How to store the children?

Child storage	Time per node	Space per node
Array of size $\|\Sigma\|+1$	$\mathcal{O}(1)$	$\Theta(\|\Sigma\|)$
List of children	$\mathcal{O}(\|\Sigma\|)$	$\Theta(\#\text{children})$
Balanced BST / AVL of children	$\mathcal{O}(\log \|\Sigma\|)$	$\Theta(\#\text{children})$
Hash table of children	$\mathcal{O}(1)$ expected	$\Theta(\#\text{children})$

In practice an array is fastest for small Σ and a hash map otherwise. Multiway tries can also be compressed.

6.5 Summary

Structure	Search time	Space
Binary search (sorted array)	$\Theta(\log n)$	$\Theta(n)$
Interpolation search	$\Theta(\log \log n)$ avg, $\Theta(n)$ worst	$\Theta(n)$
Trie (strings, length m)	$\Theta(m)$	$\mathcal{O}$(total length)
Compressed trie	$\Theta(m)$	$\Theta(n)$ nodes

- Comparison-based search needs $\Omega(\log n)$; special keys let us beat it.
- Interpolation search guesses the position from key values.
- Tries store strings by their characters; compressed tries skip unary nodes and test only the positions that distinguish keys, but must verify the full key at the leaf.

Chapter 7

Dictionaries via Hashing

7.1 Direct Addressing

If keys are integers in $\{0, \ldots, M-1\}$, store the dictionary in an array A of size M, with $A[k]$ holding the value for key k (or `nil`). `search`, `insert`, `delete` are all $\Theta(1)$, but space is $\Theta(M)$, which is wasteful when $n \ll M$ and impossible when keys come from a huge universe U.

7.2 Hashing

Definition 7.1. A *hash function* $h : U \to \{0, \ldots, M-1\}$ maps keys to slots of a *hash table* T of size M. Ideally key k is stored at $T[h(k)]$. A *collision* occurs when $h(k_1) = h(k_2)$ for $k_1 \neq k_2$.

Since $|U| \gg M$, collisions are unavoidable. Two decisions: choose a good h, and choose a collision-resolution strategy.

Definition 7.2. The *load factor* is $\alpha = n/M$.

We keep α bounded by *rehashing*: when α exceeds a threshold, allocate a table of roughly twice the size, choose a new hash function, and reinsert everything ($\Theta(n + M)$). Rehashing happens rarely enough that the amortized cost per operation is $\mathcal{O}(1)$. Similarly shrink when α gets too small, so space stays $\Theta(n)$.

Uniform hashing assumption. For analysis, assume each key is independently equally likely to hash to any slot. This is not literally true for a fixed h, which is why we later discuss universal hashing.

7.3 Separate Chaining

Each slot holds a bucket (usually an unsorted linked list) of all keys hashing to it.

- `search(k)`: scan bucket $T[h(k)]$.
- `insert(k,v)`: add to the front of bucket $T[h(k)]$: $\Theta(1)$.
- `delete(k)`: search, then unlink.

Under uniform hashing, the expected bucket length is α, so the expected cost of `search`/`delete` is $\Theta(1 + \alpha)$. With $\alpha \in \Theta(1)$ (maintained by rehashing), all operations are $\Theta(1)$ expected, while the worst case is $\Theta(n)$ (all keys in one bucket).

7.4 Open Addressing

Store all items directly in the table, at most one per slot ($\alpha \leq 1$). Each key has a *probe sequence* $h(k, 0), h(k, 1), \ldots$ of slots to try.

```
probe-insert(T, (k, v)):
  for j <- 0 to M-1 do
    if T[h(k, j)] is empty or "deleted" then
      T[h(k, j)] <- (k, v); return "success"
  return "failure to insert"       // need to rehash

probe-search(T, k):
  for j <- 0 to M-1 do
    if T[h(k, j)] is empty then return "not found"
    else if T[h(k, j)] has key k then return T[h(k, j)]
    // else: occupied by another key or "deleted": keep probing
  return "not found"
```

Deletion. We cannot simply empty a slot, since that would break probe sequences of other keys passing through it. Instead use *lazy deletion*: mark the slot as "deleted". Search continues past such slots; insert may reuse them. Rehash when too many deleted markers accumulate.

7.4.1 Linear Probing

$h(k, j) = (h(k) + j) \bmod M$. Simple and cache-friendly, but suffers from *primary clustering*: occupied runs grow, and a key hashing anywhere into a run extends it, so long runs get longer.

Example 7.3. $M = 11$, $h(k) = k \bmod 11$. Insert 41, 22, 44, 59, 32, 31, 73.

- $41 \to 8$; $22 \to 0$; $44 \to 0$ collides, goes to 1;
- $59 \to 4$; $32 \to 10$; $31 \to 9$;
- $73 \to 7$.

Now inserting 19 ($h = 8$) probes 8, 9, 10, 0, 1 and lands in 2: the clusters at 7–10 and 0–1 merged into one.

7.4.2 Quadratic Probing

$h(k, j) = (h(k) + c_1 j + c_2 j^2) \bmod M$. Avoids primary clustering but has *secondary clustering* (keys with the same $h(k)$ share a probe sequence), and must be designed carefully so the sequence visits enough slots (e.g. M prime and $\alpha < 1/2$ guarantees an empty slot is found).

7.4.3 Double Hashing

Use two independent hash functions:

$$h(k, j) = (h_0(k) + j\, h_1(k)) \bmod M,$$

where $h_1(k) \neq 0$ and is relatively prime to M (e.g. M prime) so the probe sequence is a permutation of all slots. Keys that collide at h_0 usually get different step sizes, avoiding both kinds of clustering. A common choice is the multiplicative method for h_1: $h_1(k) = \lfloor M'(kA - \lfloor kA \rfloor) \rfloor + 1$ with $A \approx (\sqrt{5} - 1)/2$.

7.4.4 Cuckoo Hashing

Use two hash functions h_0, h_1 and two tables T_0, T_1 (or one table). Every key k is stored in either $T_0[h_0(k)]$ or $T_1[h_1(k)]$ — nowhere else.

- `search` and `delete` check exactly two slots: $\Theta(1)$ *worst case.*
- `insert` places k in $T_0[h_0(k)]$; if that slot was occupied by k', then k' is kicked out and moves to its alternative slot in the other table, possibly kicking out another key, and so on.

```
cuckoo-insert(k, v):
  i <- 0
  do at most 2n times:
    if T_i[h_i(k)] is empty then
      T_i[h_i(k)] <- (k, v); return "success"
    swap (k, v) with T_i[h_i(k)]     // evict the occupant
    i <- 1 - i                       // evicted key goes to the other table
  return "failure"                   // a cycle: rehash with new h0, h1
```

If insert loops too long, there is a cycle and we rehash with new hash functions (and possibly a bigger table). With $\alpha < 1/2$ (across both tables) and random-like hash functions, the expected insertion time is $\mathcal{O}(1)$.

Example 7.4. $M = 11$, $h_0(k) = k \bmod 11$, $h_1(k) = \lfloor 11(0.618k - \lfloor 0.618k \rfloor) \rfloor$. Insert 51: $h_0 = 7$, goes to $T_0[7]$. Insert 95: $h_0 = 7$ is taken, so 95 takes $T_0[7]$ and 51 is evicted to $T_1[h_1(51)]$.

7.4.5 Expected Probes

Under uniform hashing, for open addressing with load factor $\alpha < 1$:

Strategy	Unsuccessful search / insert	Successful search
Linear probing	$\frac{1}{(1-\alpha)^2}$	$\frac{1}{1-\alpha}$
Double hashing (ideal)	$\frac{1}{1-\alpha}$	$\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$
Cuckoo hashing	$\frac{\alpha}{(1-2\alpha)^2}$ (insert)	1 (worst case ≤ 2)
Separate chaining	$1 + \alpha$	$1 + \alpha/2$

(Up to constants and lower-order terms.) The key point: all are $\mathcal{O}(1)$ when α is bounded away from the critical value (1 for probing, 1/2 for cuckoo).

Sketch for ideal double hashing. If each probe is to a uniformly random slot not yet probed, each probe finds an empty slot with probability at least $1 - \alpha$. The number of probes is then dominated by a geometric random variable with mean $\frac{1}{1-\alpha}$. □

7.5 Choosing a Hash Function

Goals: fast to compute and spreads keys uniformly, even for structured (non-random) keys.

- **Modular method:** $h(k) = k \bmod M$ with M prime, not close to a power of 2 (otherwise h depends only on low-order bits).
- **Multiplicative method:** $h(k) = \lfloor M(kA - \lfloor kA \rfloor) \rfloor$ for a constant $0 < A < 1$; Knuth suggests $A = \varphi - 1 \approx 0.618$. Here M can be any size.

- **Strings:** interpret $s = s_0 s_1 \dots s_{p-1}$ as a number in base R (e.g. 255) and reduce mod M, using Horner's rule: $h(s) = (\cdots((s_0 R + s_1)R + s_2)\cdots) \bmod M$, reducing at each step.

7.5.1 Universal Hashing

For any fixed h, an adversary can choose n keys that all collide, forcing $\Theta(n)$ per operation. The fix is randomization: choose h at random from a family when the table is created (and at each rehash).

Definition 7.5. A family $\mathcal{H}$ of hash functions $U \to \{0, \dots, M-1\}$ is *universal* if for every pair of distinct keys $k_1 \neq k_2$,

$$\Pr_{h \in \mathcal{H}}[h(k_1) = h(k_2)] \leq \frac{1}{M}.$$

Theorem 7.6. *Let $p > |U|$ be prime and $\mathcal{H} = \{h_{a,b}(k) = ((ak+b) \bmod p) \bmod M : 1 \leq a < p,\ 0 \leq b < p\}$. Then $\mathcal{H}$ is universal.*

Theorem 7.7. *With h drawn from a universal family and separate chaining, the expected length of the bucket for any key k is at most $1 + \alpha$. Thus every operation takes $\mathcal{O}(1 + \alpha)$ expected time for* every *input.*

Proof. For each other key k' in the table, let $X_{k'}$ be 1 if $h(k') = h(k)$. $\mathbb{E}[X_{k'}] \leq 1/M$. By linearity, the expected number of colliding keys is $\leq n/M = \alpha$; add 1 for k itself. □

7.6 Hashing vs. Balanced Search Trees

	Hash table	AVL tree
search/insert/delete	$\mathcal{O}(1)$ expected	$\Theta(\log n)$ worst case
worst case	$\Theta(n)$ (cuckoo: $\mathcal{O}(1)$ search)	$\Theta(\log n)$
ordered operations (min, successor, range)	not supported	supported
space	wasted empty slots	pointers + heights

7.7 Summary

- Direct addressing is $\Theta(1)$ but needs space $\Theta(|U|)$.
- Hashing maps a large universe to M slots; keep $\alpha = n/M$ bounded by rehashing (amortized $\mathcal{O}(1)$).
- Chaining: expected $\Theta(1 + \alpha)$.
- Open addressing: linear probing (primary clustering), quadratic probing (secondary clustering), double hashing; lazy deletion.
- Cuckoo hashing: $\mathcal{O}(1)$ worst-case search and delete; expected $\mathcal{O}(1)$ insert when $\alpha < 1/2$.
- Universal hashing gives expected guarantees for every input.

Chapter 8

Range Searching and Multidimensional Data

8.1 Range Queries

Definition 8.1. A *range query* `RangeSearch(x1, x2)` asks for all KVPs with key k satisfying $x_1 \leq k \leq x_2$. The output size is denoted s.

Hash tables cannot answer range queries efficiently (they scatter nearby keys). Sorted arrays and balanced BSTs can.

- **Sorted array:** binary search for x_1 and x_2, report what lies between: $\mathcal{O}(\log n + s)$.
- **BST:** described next: $\mathcal{O}(h + s)$.

8.1.1 Range Search in a BST

```
BST-RangeSearch(T, x1, x2):
  if T is empty then return
  if T.key < x1 then BST-RangeSearch(T.right, x1, x2)
  else if T.key > x2 then BST-RangeSearch(T.left, x1, x2)
  else
    BST-RangeSearch(T.left, x1, x2)
    report T.key
    BST-RangeSearch(T.right, x1, x2)
```

Classify the nodes visited: perform a BST search for x_1 (path P_1) and for x_2 (path P_2). Then each visited node is

- a *boundary node* (on P_1 or P_2): $\mathcal{O}(h)$ of these;
- an *inside node* (between the paths): all are reported, so $\leq s$;
- outside nodes are never visited, apart from children of boundary nodes.

So the time is $\mathcal{O}(h + s)$, i.e. $\mathcal{O}(\log n + s)$ in an AVL tree.

8.2 Multidimensional Data

Now each item is a point in $\mathbb{R}^d$ (e.g. a laptop with price, weight, screen size). An *orthogonal range query* is an axis-aligned box $[x_1, x_1'] \times \cdots \times [x_d, x_d']$; report all points in it. We mostly study $d = 2$. Assume for simplicity that points are in *general position* (no two share an x- or y-coordinate), or handle ties carefully.

Naive: scan all points, $\Theta(n)$. Or use d separate 1D structures and intersect: can cost $\Theta(n)$ even when s is tiny.

8.3 Quadtrees

Assume points lie in a square bounding box $R = [0, 2^k) \times [0, 2^k)$.

Definition 8.2. A *quadtree* recursively splits a region into four equal quadrants NE, NW, SW, SE. A region with at most one point is a leaf (storing that point or empty); otherwise it is an internal node with four children.

- `search`: descend into the quadrant containing the point.
- `insert`: search to a leaf; if it already holds a point, split repeatedly until the two points are in different quadrants.
- `delete`: remove the point; if a node then has at most one point in its subtree, collapse it into a leaf.

```
QTree-RangeSearch(v, A):     // v: node with region R(v), A: query box
  if R(v) is disjoint from A then return
  if R(v) is inside A then report all points below v; return
  if v is a leaf then
    if v's point is in A then report it
    return
  for each child w of v do QTree-RangeSearch(w, A)
```

Analysis. The height is not bounded in terms of n; it depends on the *spread factor* $\beta = \frac{\text{side length of } R}{\text{min distance between two points}}$: height $h \in \Theta(\log \beta)$. Building takes $\Theta(nh)$; range search can take $\Theta(nh)$ in the worst case even if $s = 0$. Two points very close together force a long chain.

Pros: very simple, easy to generalise to d dimensions (octrees for $d = 3$), good for images (each pixel region) and uniform data. Cons: wasted space, height depends on geometry.

8.4 kd-Trees

Quadtrees split the *space* evenly; kd-trees split the *points* evenly.

Definition 8.3. A *kd-tree* (2D) is built by splitting the point set at the median x-coordinate into two halves by a vertical line, then splitting each half at its median y-coordinate by a horizontal line, and so on, alternating directions. Each internal node stores its splitting line; leaves store single points.

```
build-kdtree(P, dir):
  if |P| <= 1 then return leaf(P)
  m <- median of P by coordinate dir        // QuickSelect: Theta(|P|) expected
  P_low  <- points with coordinate <= m
  P_high <- points with coordinate >  m
  v <- node with split line (dir = m)
  v.left  <- build-kdtree(P_low,  other(dir))
  v.right <- build-kdtree(P_high, other(dir))
  return v
```

Build time $T(n) = 2T(n/2) + \Theta(n) \in \Theta(n \log n)$ expected (or by presorting). Height $\Theta(\log n)$ (assuming general position), space $\Theta(n)$.

Range search is the same as for quadtrees: each node has a region; skip disjoint regions, report whole subtrees for contained regions, recurse otherwise.

Theorem 8.4. *Range search in a 2D kd-tree takes $\mathcal{O}(\sqrt{n} + s)$ time.*

Proof sketch. Time is $\mathcal{O}(s + Q(n))$ where $Q(n)$ counts nodes whose region intersects the boundary of the query box without being contained in it. Any such region is crossed by one of the four sides of the box; consider an axis-parallel line ℓ. Going down two levels, a node's region splits into four grandchildren, and a vertical (say) line can intersect at most two of them. So $Q(n) \leq 2Q(n/4) + \mathcal{O}(1)$, which solves to $Q(n) \in \mathcal{O}(\sqrt{n})$. □

In d dimensions: build $\mathcal{O}(n \log n)$, space $\mathcal{O}(n)$, range query $\mathcal{O}(n^{1-1/d} + s)$.

8.5 Range Trees

To get polylogarithmic queries, trade space.

Definition 8.5. A 2D *range tree* consists of a balanced BST T (the *primary structure*) on the x-coordinates, where every node v additionally stores an *associated structure* $T(v)$: a balanced BST on the y-coordinates of all points in v's subtree.

Space. Each point appears in the associated structure of each of its $\mathcal{O}(\log n)$ ancestors, so total space is $\mathcal{O}(n \log n)$.

Range search for $[x_1, x_2] \times [y_1, y_2]$:

1. Search for x_1 and x_2 in T, giving boundary paths P_1, P_2. Check each boundary node individually: $\mathcal{O}(\log n)$.
2. The *topmost inside nodes* are the children of boundary nodes that lie strictly between the paths: $\mathcal{O}(\log n)$ of them, and their subtrees cover exactly the points with x in range (apart from boundary nodes).
3. For each topmost inside node v, run a 1D range search for $[y_1, y_2]$ in $T(v)$: $\mathcal{O}(\log n + s_v)$.

```
RangeTree-Search(T, x1, x2, y1, y2):
  P1 <- search path for x1;  P2 <- search path for x2
  for each node v on P1 or P2 do
    if v's point is in the box then report it
```

```
  for each topmost inside node v do
    BST-RangeSearch(T(v), y1, y2)
```

Total query time $\mathcal{O}(\log^2 n + s)$. Build in $\mathcal{O}(n \log n)$ by building associated trees bottom-up from sorted lists. (With fractional cascading, a CS 240 aside, the query drops to $\mathcal{O}(\log n + s)$.)

Insert/delete are awkward: the primary tree cannot be rotated cheaply because rotation changes associated structures. Instead allow imbalance and rebuild subtrees occasionally.

In d dimensions: space $\mathcal{O}(n \log^{d-1} n)$, query $\mathcal{O}(\log^d n + s)$.

8.6 Comparison

Structure	Space	Build	2D range query
Quadtree	$\Theta(nh)$	$\Theta(nh)$	$\Theta(nh)$ worst
kd-tree	$\Theta(n)$	$\Theta(n \log n)$	$\mathcal{O}(\sqrt{n} + s)$
Range tree	$\Theta(n \log n)$	$\Theta(n \log n)$	$\mathcal{O}(\log^2 n + s)$

Here $h \in \Theta(\log \beta)$ for the quadtree.

8.7 Summary

- 1D range search in a balanced BST: $\mathcal{O}(\log n + s)$ via boundary/inside node classification.
- Quadtrees: split space into quadrants; simple but height depends on spread.
- kd-trees: alternate median splits; linear space, $\mathcal{O}(\sqrt{n} + s)$ queries.
- Range trees: BST of BSTs; $\mathcal{O}(n \log n)$ space, $\mathcal{O}(\log^2 n + s)$ queries.

Chapter 9

String Matching

9.1 The Problem

Definition 9.1. *Pattern matching*: given a *text* $T[0..n-1]$ and a *pattern* $P[0..m-1]$ over an alphabet Σ, find the first index i such that $T[i..i+m-1] = P$ (or report all such i, or "FAIL").

Terminology: a *substring* $T[i..j]$; a *prefix* $T[0..i]$; a *suffix* $T[i..n-1]$. A *guess* (or shift) is a position i where P might start; a *check* compares one character $T[i+j]$ with $P[j]$. Algorithms differ in which guesses they skip and how they order checks.

9.2 Brute Force

```
brute-force(T, P):
  for i <- 0 to n - m do
    j <- 0
    while j < m and T[i+j] = P[j] do j <- j + 1
    if j = m then return i
  return FAIL
```

Worst case $\Theta((n-m+1)m)$, i.e. $\Theta(nm)$ for $m \le n/2$; e.g. $T = a^n$, $P = a^{m-1}b$. The waste: after a mismatch we forget everything we learned about T.

9.3 Knuth–Morris–Pratt (KMP)

Idea: on a mismatch at $P[j]$, we know $T[i..i+j-1] = P[0..j-1]$. Shift the pattern so that the longest proper prefix of P that is also a suffix of $P[0..j-1]$ lines up, and never move backwards in T.

Definition 9.2. The *failure function* $F[j]$ is the length of the longest prefix of P that is a suffix of $P[1..j]$; equivalently, the longest proper suffix of $P[0..j]$ that is also a prefix of P.

Example 9.3. $P =$ `abacaba`:

j	0	1	2	3	4	5	6
$P[j]$	a	b	a	c	a	b	a
$F[j]$	0	0	1	0	1	2	3

E.g. $F[6] = 3$ because `aba` is both a prefix of P and a suffix of `abacaba` (proper).

```
KMP(T, P):
  F <- failure-array(P)
  i <- 0; j <- 0                 // i indexes T, j indexes P
  while i < n do
    if T[i] = P[j] then
      if j = m - 1 then return i - j      // match
      i <- i + 1; j <- j + 1
    else if j > 0 then
      j <- F[j - 1]              // shift pattern, keep i
    else
      i <- i + 1
  return FAIL

failure-array(P):
  F[0] <- 0
  i <- 1; j <- 0
  while i < m do
    if P[i] = P[j] then
      j <- j + 1; F[i] <- j; i <- i + 1
    else if j > 0 then
      j <- F[j - 1]
    else
      F[i] <- 0; i <- i + 1
  return F
```

The failure array is computed by running KMP on P against itself.

Theorem 9.4. *KMP runs in $\Theta(n)$ time after $\Theta(m)$ preprocessing.*

Proof. Consider the potential $2i - j$. Each loop iteration either increases i (and j by at most 1), raising $2i - j$ by at least 1, or decreases j (since $F[j-1] < j$) with i fixed, again raising $2i - j$ by at least 1. Since $2i - j \le 2n$, there are at most $2n$ iterations. The same argument gives $\mathcal{O}(m)$ for the failure array. □

9.4 Boyer–Moore

Boyer–Moore compares the pattern *right to left*, and uses two heuristics to make large jumps:

- **Bad character (last-occurrence) heuristic:** on a mismatch with text character $c = T[i]$, shift so that the last occurrence of c in P lines up with $T[i]$; if c does not occur in P, shift P entirely past $T[i]$.
- **Good suffix heuristic:** if a suffix $P[j+1..m-1]$ matched, shift to align it with its previous occurrence in P (similar to KMP's failure function, but for suffixes).

The shift used is the larger of the two. In the course we mainly trace the bad character rule.

Definition 9.5. The *last-occurrence function* $L(c)$ is the largest index j with $P[j] = c$, or -1 if c does not occur in P.

Example 9.6. $P =$ `abacab`, $\Sigma = \{a, b, c, d\}$: $L(a) = 4$, $L(b) = 5$, $L(c) = 3$, $L(d) = -1$.

```
boyer-moore(T, P):
  L <- last-occurrence array of P      // Theta(m + |Sigma|)
  S <- good-suffix array of P          // Theta(m)
  i <- m - 1; j <- m - 1               // i in T, j in P
  while i < n do
    if T[i] = P[j] then
      if j = 0 then return i           // match
      i <- i - 1; j <- j - 1
    else
      i <- i + m - 1 - min(L[T[i]], S[j])
      j <- m - 1
  return FAIL
```

With only the bad character rule, the update is $i \leftarrow i + m - \min(j, 1 + L(T[i]))$, which always shifts the pattern forward by at least one.

Performance. Worst case $\mathcal{O}(nm)$ for the simple version (e.g. $T = a^n$, $P = ba^{m-1}$ with bad character only); with the full good suffix rule plus an extra trick (Galil's rule) it becomes $\mathcal{O}(n + m + |\Sigma|)$. In practice on English text it is the fastest: it typically checks only a fraction of the text characters, about n/m for random text over a large alphabet.

9.5 Rabin–Karp Fingerprinting

Idea: use hashing to rule out most guesses with one comparison of numbers. Compute $h(P)$ and compare it with $h(T[i..i+m-1])$ for each i; only check character by character when the hashes are equal.

To make this efficient, use a *rolling hash*: interpret strings as base-R numbers modulo a prime M:

$$h(T[i..i+m-1]) = \Big(\sum_{j=0}^{m-1} T[i+j] \cdot R^{m-1-j} \Big) \bmod M.$$

Then the next window's hash is computed in $\mathcal{O}(1)$:

$$h_{i+1} = \big((h_i - T[i] \cdot R^{m-1}) \cdot R + T[i+m] \big) \bmod M.$$

```
rabin-karp(T, P):
  M <- a suitable (random) prime
  hP <- h(P[0..m-1]); hT <- h(T[0..m-1])
  s <- R^(m-1) mod M
  for i <- 0 to n - m do
    if hT = hP then
      if T[i..i+m-1] = P then return i     // verify (false positives possible)
    if i < n - m then
      hT <- ((hT - T[i]*s) * R + T[i+m]) mod M
  return FAIL
```

Example 9.7. $R = 10$, $M = 97$, $P = 59265$: $h(P) = 59265 \bmod 97 = 95$. For $T = 31415926535\ldots$, the first window $31415 \bmod 97 = 84 \neq 95$; the next window's hash is

$$h = \big((84 - 3 \cdot (10^4 \bmod 97)) \cdot 10 + 9 \big) \bmod 97,$$

and so on.

Analysis. Expected time $\mathcal{O}(n+m)$ if M is a random prime large enough that false positives are rare (probability about $1/M$ per window). Worst case $\Theta(nm)$ (every window is a false positive). Nice property: it generalises easily to multiple patterns and 2D matching.

9.6 Suffix Tries and Suffix Trees

The previous algorithms preprocess the *pattern*. If the text is fixed and we search many patterns, preprocess the *text* instead.

Lemma 9.8. *P occurs in T iff P is a prefix of some suffix of T.*

Definition 9.9. The *suffix trie* of T is the trie of all $n+1$ suffixes of $T\$$. A *suffix tree* is the compressed (Patricia) trie of these suffixes. Leaves store the start index of their suffix; edge labels are stored as index pairs into T rather than as strings.

- A suffix trie can have $\Theta(n^2)$ nodes.
- A suffix tree has $n+1$ leaves and at most n internal nodes, so $\Theta(n)$ space (plus the text itself).
- Construction: naively $\Theta(n^2)$ by inserting suffixes one by one; $\Theta(n)$ with Ukkonen's or McCreight's algorithm (beyond CS 240).
- Search for P: walk down from the root comparing characters of P with edge labels: $\mathcal{O}(m)$ (with $\mathcal{O}(|\Sigma|)$ or $\mathcal{O}(1)$ per child lookup). If P is exhausted, any leaf below the current point gives an occurrence; reporting all k occurrences takes $\mathcal{O}(m+k)$.

Example 9.10. $T =$ `bananaban`. The suffixes include `bananaban$`, `ananaban$`, `nanaban$`, …, `n$`, `$`. Searching `ana` walks down the path spelling `a`, `n`, `a` (some of these letters lie inside compressed edges) and ends in a subtree whose leaves are the suffixes starting at 1 and 3: two occurrences.

9.7 Suffix Arrays

Suffix trees use a lot of memory in practice (pointers per node). A *suffix array* stores only the sorted order of the suffixes.

Definition 9.11. The *suffix array* A^s of T is the array of starting indices of the suffixes of $T\$$, sorted lexicographically ($ smallest).

Example 9.12. $T =$ `banana$`:

j	$A^s[j]$	suffix
0	6	`$`
1	5	`a$`
2	3	`ana$`
3	1	`anana$`
4	0	`banana$`
5	4	`na$`
6	2	`nana$`

Construction. Sorting suffixes directly with MSD radix sort costs $\mathcal{O}(n^2)$ worst case. The course's approach: repeated doubling. Sort by the first character, then by the first 2, 4, 8, ... characters, where the sort key for length 2ℓ is the pair (rank of first ℓ chars, rank of the next ℓ chars), sorted by radix sort in $\mathcal{O}(n)$. There are $\log n$ rounds: $\mathcal{O}(n \log n)$ total.

Search. The occurrences of P form a contiguous block of A^s. Binary search, comparing P with the suffix at the midpoint ($\mathcal{O}(m)$ per comparison):

```
suffix-array-search(T, A, P):
  l <- 0; r <- n - 1
  while l <= r do
    v <- floor((l + r) / 2)
    i <- A[v]
    compare P with T[i..i+m-1]
    if P is a prefix of T[i..] then return i
    else if P < T[i..i+m-1] then r <- v - 1
    else l <- v + 1
  return FAIL
```

Time $\mathcal{O}(m \log n)$; finding all k occurrences takes $\mathcal{O}(m \log n + k)$.

9.8 Summary

Algorithm	Preprocessing	Search time	Extra space
Brute force	–	$\mathcal{O}(nm)$	$\mathcal{O}(1)$
KMP	$\mathcal{O}(m)$	$\mathcal{O}(n)$	$\mathcal{O}(m)$
Boyer–Moore	$\mathcal{O}(m + \lvert\Sigma\rvert)$	$\mathcal{O}(n)$ (often sublinear)	$\mathcal{O}(m + \lvert\Sigma\rvert)$
Rabin–Karp	$\mathcal{O}(m)$	$\mathcal{O}(n + m)$ expected	$\mathcal{O}(1)$
Suffix tree	$\mathcal{O}(n)$ (or n^2 naive) on T	$\mathcal{O}(m)$	$\mathcal{O}(n)$
Suffix array	$\mathcal{O}(n \log n)$ on T	$\mathcal{O}(m \log n)$	$\mathcal{O}(n)$

- KMP never moves backward in T; its failure function encodes self-overlap.
- Boyer–Moore scans right to left and jumps using last occurrences.
- Rabin–Karp filters guesses with a rolling hash.
- Suffix trees/arrays preprocess the text so each pattern search is fast.

Chapter 10

Compression

10.1 Encoding Basics

Definition 10.1. We transform a *source text* S over alphabet Σ_S into a *coded text* C over alphabet Σ_C (usually $\{0,1\}$). *Encoding* maps $S \mapsto C$; *decoding* maps $C \mapsto S$. Compression is *lossless* if S is recovered exactly (all methods in this chapter).

Definition 10.2. The *compression ratio* is $\dfrac{|C| \cdot \log|\Sigma_C|}{|S| \cdot \log|\Sigma_S|}$. Smaller is better.

No lossless scheme can compress *every* input: there are 2^n bit strings of length n but only $2^n - 1$ shorter ones (pigeonhole). Compression exploits structure: skewed character frequencies (Huffman), long runs (RLE), repeated substrings (LZW), or rearrangements that create such structure (BWT, MTF).

10.1.1 Character Encodings

A *fixed-length code* (like ASCII, 7 bits per character) is easy to decode. A *variable-length code* gives frequent characters shorter codewords (e.g. Morse code). To decode unambiguously without separators, use a *prefix-free* code: no codeword is a prefix of another. A prefix-free binary code corresponds to a binary *trie* whose leaves are the characters; decoding walks the trie from the root, emitting a character at each leaf.

```
prefix-free-decode(C, trie):
  S <- empty; i <- 0
  while i < |C| do
    r <- trie.root
    while r is not a leaf do
      if i = |C| then return "invalid encoding"
      r <- child of r labelled C[i]; i <- i + 1
    append r's character to S
  return S
```

10.2 Huffman Coding

Given frequencies $f(c)$ of each character in S, the encoded length using trie T is $\sum_c f(c) \cdot \text{depth}_T(c)$. Huffman's algorithm finds a trie minimising this.

```
Huffman-encoding(S):
  compute frequencies f(c) for each c in S
  Q <- min-heap of single-node tries, keyed by frequency
  while Q.size > 1 do
    T1 <- Q.deleteMin(); T2 <- Q.deleteMin()
    T <- new trie with children T1, T2, weight f(T1) + f(T2)
    Q.insert(T)
  T <- Q.deleteMin()
  C <- encode S using the codewords from T
  return C and T          // the trie must be sent too
```

Example 10.3. $S =$ `LOSSLESS`: $f(S) = 4$, $f(L) = 2$, $f(O) = 1$, $f(E) = 1$. Merge O and E (weight 2); merge L with that (weight 4); merge S with that (weight 8). One resulting code: S = `0`, L = `10`, O = `110`, E = `111`. Encoded length: $4 \cdot 1 + 2 \cdot 2 + 1 \cdot 3 + 1 \cdot 3 = 14$ bits, versus 16 for a fixed 2-bit code.

Analysis. With $|\Sigma_S| = \sigma$ distinct characters, building the trie costs $\mathcal{O}(\sigma \log \sigma)$, and encoding/decoding costs $\mathcal{O}(|S| + |C|)$ (plus $\mathcal{O}(|S|)$ to count frequencies).

Theorem 10.4. *The Huffman trie minimises $\sum_c f(c)\,\mathit{depth}(c)$ among all prefix-free binary codes.*

Proof sketch. By induction on σ. In some optimal trie, the two least frequent characters x, y are siblings at maximum depth (exchange argument: swapping them with deepest siblings cannot increase cost). Replacing x, y by a single character z with $f(z) = f(x) + f(y)$ reduces the cost of every such trie by exactly $f(x) + f(y)$; by induction, Huffman is optimal for the reduced alphabet, so it is optimal for the original. □

Drawbacks: two passes over the text, the trie must be transmitted, and the result is not unique (ties). It is optimal only among codes that encode one character at a time.

10.3 Run-Length Encoding

Useful when the source has long runs of identical characters (e.g. bitmaps, fax). For a binary S: output the first bit, then the lengths of successive runs, each encoded with a prefix-free code for integers.

Definition 10.5. The *Elias gamma code* of a positive integer k is $\lfloor \log_2 k \rfloor$ zeros followed by the binary representation of k (which has $\lfloor \log_2 k \rfloor + 1$ bits). E.g. $1 \mapsto$ `1`, $3 \mapsto$ `011`, $5 \mapsto$ `00101`, $10 \mapsto$ `0001010`.

```
RLE-encoding(S):            // S a bit string
  C <- S[0]
  i <- 0
  while i < |S| do
    k <- 1
    while i + k < |S| and S[i+k] = S[i] do k <- k + 1
    append Elias-gamma(k) to C
    i <- i + k
  return C
```

Example 10.6. $S =$ `00000111000`: first bit 0, runs 5, 3, 3, so $C =$ `0 00101 011 011` (12 bits vs. 11 — no gain here). For $S = 0^{100}$, C is $1 + 13 = 14$ bits.

A run of length k costs $2\lfloor \log k \rfloor + 1$ bits, so RLE wins when runs are long ($k \geq 6$ or so) and can *expand* text with short runs (runs of length 2 or 4 cost 3 or 5 bits).

10.4 Lempel–Ziv–Welch (LZW)

Huffman and RLE ignore repeated *substrings*. LZW is *adaptive*: it builds a dictionary of substrings on the fly, and the decoder rebuilds the same dictionary, so nothing extra is transmitted.

Encoding. Start with a dictionary D of all single characters (e.g. ASCII codes 0–127). Repeatedly find the longest prefix w of the remaining text that is in D, output its code, and add w followed by the next character to D with the next unused code (128, 129, ...). The dictionary is stored as a trie, so finding w is a walk down the trie.

```
LZW-encode(S):
  D <- trie with all ASCII chars (codes 0..127)
  idx <- 128
  while S is non-empty do
    v <- root of D; K <- first char of S
    while v has a child labelled K do
      v <- that child; remove K from front of S
      if S is empty then break
      K <- first char of S
    output code(v)
    if S is non-empty then
      add child of v labelled K with code idx
      idx <- idx + 1
```

Decoding. Read codes one by one, maintaining the same dictionary (as an array code $\to$ string). After decoding code c to string s, add $s_{\text{prev}} + s[0]$ to the dictionary. Special case: the code may not yet be in the dictionary (it was created by the encoder in the immediately preceding step); then it must equal $s_{\text{prev}} + s_{\text{prev}}[0]$.

```
LZW-decode(C):
  D <- array with ASCII chars at codes 0..127; idx <- 128
  code <- first code of C; s <- D[code]; output s
  while there are more codes do
    sprev <- s
    code <- next code of C
    if code < idx then s <- D[code]
    else s <- sprev + sprev[0]           // the tricky case (code = idx)
    output s
    D[idx] <- sprev + s[0]; idx <- idx + 1
```

Example 10.7. Encode $S =$ `ABABABA` (A = 65, B = 66):

Longest match w	Output	Added	Code
A	65	AB	128
B	66	BA	129
AB	128	ABA	130
ABA	130	–	–

Output: 65, 66, 128, 130. Decoding: 65 → A; 66 → B (add AB = 128); 128 → AB (add BA = 129); 130 is not yet known, so it is AB + A = ABA (add ABA = 130). Result ABABABA.

Both directions run in $\mathcal{O}(|S|)$ time. In practice codes are fixed-width (e.g. 12 bits), and the dictionary is reset or frozen when full. LZW (and relatives LZ77/LZ78) underlie GIF, `compress`, and (via LZ77) `gzip`.

10.5 Move-To-Front Transform

Recall the MTF heuristic for self-organising lists (Chapter 5). As a transform: keep a list of the alphabet; for each character of S, output its current index in the list and move it to the front.

```
MTF-encode(S):
  L <- array of Sigma_S in a fixed order
  for each c in S do
    i <- index of c in L
    output i
    move c to the front of L (shifting L[0..i-1] right)
```

Decoding is symmetric: output $L[i]$, then move it to the front. Runs of the same character become runs of 0s, and recently used characters get small numbers; the output (over the alphabet $\{0, \ldots, \sigma - 1\}$) is then well suited to Huffman or RLE. MTF alone does not compress.

10.6 Burrows–Wheeler Transform

The BWT is a reversible permutation of the text that tends to group equal characters together, making it highly compressible by MTF + RLE/Huffman.

Encoding. Append the end-marker \$ (smallest character). Write down all n cyclic shifts of S, sort them lexicographically, and output the *last column*.

Example 10.8. $S =$ `banana$`. Sorted cyclic shifts:

```
$banana
a$banan
ana$ban
anana$b
banana$
na$bana
nana$ba
```

$C =$ `annb$aa`. Note the first column is the sorted text, and the cyclic shifts are in the same order as the suffix array $[6, 5, 3, 1, 0, 4, 2]$: the BWT is $C[j] = S[A^s[j] - 1]$ (cyclically). Computing it via a suffix array takes $\mathcal{O}(n \log n)$.

Why it helps. Shifts starting with the same context (e.g. `he_`) are sorted together, and the character preceding a common context is often the same (e.g. `t` before `he_`), so the last column has long runs.

Decoding. The first column is C sorted. Each row is a cyclic shift, so the last character of a row precedes its first character in S. Key fact: the i-th occurrence of a character c in the last column corresponds to the i-th occurrence of c in the first column (they are the same text position).

```
BWT-decode(C):
  A <- array of pairs (C[i], i) for i = 0..n-1
  stably sort A by character       // A[j] = (j-th char of first column,
                                   //   row where that same occurrence sits in C)
  j <- index with C[j] = $         // the row equal to S itself ends in $
  S <- empty
  repeat
    (c, k) <- A[j]
    append c to S
    j <- k
  until c = $
  return S
```

In words: row j of the sorted matrix starts with the character $A[j].c$; the row that starts at the next text position is the one whose *last* character is this same occurrence, namely row $A[j].k$. Stability of the sort is exactly what matches the i-th c of the first column with the i-th c of the last column.

Example 10.9. Decode $C =$ `annb$aa`. Stably sorting the pairs gives

$$A = [(\$, 4), (a, 0), (a, 5), (a, 6), (b, 3), (n, 1), (n, 2)].$$

$C[4] = \$$, so start with $j = 4$: $A[4] = (b, 3)$ outputs `b`, go to 3; $A[3] = (a, 6)$ outputs `a`; $A[6] = (n, 2)$: `n`; $A[2] = (a, 5)$: `a`; $A[5] = (n, 1)$: `n`; $A[1] = (a, 0)$: `a`; $A[0] = (\$, 4)$: `$`, stop. Result: `banana$`.

Decoding runs in $\mathcal{O}(n)$ time (counting sort for the stable sort). The `bzip2` pipeline is: BWT $\to$ MTF $\to$ RLE $\to$ Huffman.

10.7 Summary

Method	Exploits	Notes
Huffman	skewed char frequencies	optimal prefix code; 2 passes; send trie
RLE	long runs	Elias gamma for run lengths; can expand
LZW	repeated substrings	adaptive, 1 pass, dictionary not sent
MTF	locality of characters	transform only; output has small numbers
BWT	repeated contexts	transform only; reversible; groups chars

- Prefix-free codes are tries; decode by walking from the root.
- Huffman builds the optimal trie greedily with a min-heap.
- LZW's decoder rebuilds the dictionary, with one special case.
- BWT sorts cyclic shifts and outputs the last column; decoding uses the last-to-first correspondence of equal characters.

Chapter 11

External Memory

11.1 The External Memory Model

When data is too large for main memory it lives on disk (or SSD). Accessing a disk is many orders of magnitude slower than accessing RAM, but data is transferred in large *blocks*, so reading a whole block costs about the same as reading one word of it.

Definition 11.1 (External memory model (EMM))**.** There is an internal memory of size M and an unbounded external memory. Data moves between them in blocks of B consecutive items. The cost of an algorithm is the number of *block transfers* (reads and writes); computation in internal memory is free.

The goal: design algorithms whose block transfers are a factor B (or better) less than their RAM operation counts, by using every item of each block we read.

Example 11.2. Scanning an array of n items costs $\Theta(n/B)$ transfers. Binary search on a sorted array costs $\Theta(\log(n/B))$ transfers, barely better than $\log n$, because successive probes are in different blocks. A balanced BST with nodes scattered on disk costs $\Theta(\log n)$ transfers per search.

11.2 External Sorting

HeapSort and QuickSort access memory all over the place: $\Theta(n \log n)$ transfers in the worst case. MergeSort is naturally sequential, and adapts well.

Merging. Merging two sorted runs of total length ℓ needs three blocks of internal memory (two inputs, one output) and $\Theta(\ell/B)$ transfers.

d-way merge. Merge d runs at once: keep one block per run plus an output block, and a min-heap of the d current front items to pick the next smallest. Requires $(d+1)B \le M$, so take $d \approx M/B - 1$.

```
d-way-merge(S1, ..., Sd):          // each Si sorted, read block by block
  P <- min-heap of (front item of Si, i) for i = 1..d
  S <- empty output (written block by block)
  while P is not empty do
    (x, i) <- P.deleteMin()
    append x to S
```

```
    remove x from Si
    if Si is not empty then P.insert((front of Si, i))
  return S
```

External MergeSort.

1. Create initial runs: read M items at a time, sort them internally, write them back. This gives n/M sorted runs, costing $\Theta(n/B)$ transfers.
2. Repeatedly d-way merge groups of $d \approx M/B$ runs. Each round reads and writes everything once ($\Theta(n/B)$ transfers) and divides the number of runs by d.

Number of rounds: $\log_d(n/M)$. Total transfers:

$$\Theta\left(\frac{n}{B}\log_{M/B}\frac{n}{M}\right).$$

This is optimal: any comparison-based external sort needs $\Omega(\frac{n}{B}\log_{M/B}\frac{n}{B})$ transfers.

Example 11.3. With $n = 10^{10}$, $M = 10^8$, $B = 10^4$: initial runs give 100 runs; one merge round with $d = M/B = 10^4 \geq 100$ suffices. So the whole sort reads and writes the data only twice.

11.3 External Dictionaries: B-Trees

To make search efficient in the EMM we want trees whose nodes fill a block and have many children, reducing the height from $\log_2 n$ to $\log_B n$.

11.3.1 Multiway Search Trees

A node of a *multiway search tree* stores k sorted keys $k_1 < \cdots < k_k$ and has $k+1$ subtrees $T_0, \ldots, T_k$, where all keys in T_i lie between k_i and k_{i+1} (with $k_0 = -\infty$, $k_{k+1} = +\infty$). Search: at each node find the subtree to descend into (inside the node this is free in the EMM, or $\mathcal{O}(\log k)$ by binary search).

11.3.2 2-3 Trees

Definition 11.4. A *2-3 tree* is a multiway search tree in which

- every internal node has either 1 key and 2 children, or 2 keys and 3 children;
- all empty subtrees (leaves) are at the same depth.

Insert. Search for k; it ends at the lowest internal level. Add k to that node. If the node now has 3 keys (*overflow*), *split* it: the middle key moves up into the parent, and the left and right keys become two separate nodes. The parent may overflow in turn; if the root splits, a new root is created and the tree grows by one level (the only way height increases).

Example 11.5. Insert 10, 20, 30 into an empty 2-3 tree: [10, 20] then [10, 20, 30] overflows; split: root [20] with children [10] and [30]. Insert 40, 50: [30, 40] then [30, 40, 50] overflows; 40 moves up: root [20, 40] with children [10], [30], [50].

Delete. If k is in an internal node, swap it with its successor (in a bottom node) first. Remove k from the bottom node. If the node becomes empty (*underflow*):

- if an immediate sibling has 2 keys, *transfer* (rotate): a key moves from the sibling up to the parent and the parent's separating key moves down;
- otherwise *merge* with a sibling, pulling the separating key down from the parent; the parent may underflow in turn. If the root becomes empty, delete it (height decreases).

11.3.3 (a,b)-Trees

Definition 11.6. For $2 \le a \le \lceil b/2 \rceil$, an (a, b)-*tree* is a multiway search tree in which

- every non-root internal node has between a and b children (so between $a-1$ and $b-1$ keys);
- the root has between 2 and b children;
- all leaves (empty subtrees) are at the same depth.

A 2-3 tree is a $(2, 3)$-tree; a $(2, 4)$-tree is also called a 2-3-4 tree.

The condition $a \le \lceil b/2 \rceil$ makes splitting work: an overflowing node has b keys; splitting it around the median yields two nodes with at least $\lceil b/2 \rceil - 1 \ge a - 1$ keys each.

Theorem 11.7. *An (a, b)-tree with n keys has height $\Theta(\log_a n)$; more precisely $h \in \mathcal{O}(\log_a n)$ and $h \in \Omega(\log_b n)$.*

Proof. Count nodes at each level: the root has at least 2 children, and each other internal node at least a. So level $i \ge 1$ has at least $2a^{i-1}$ nodes, and the number of leaves (empty subtrees) is at least $2a^{h-1}$. A multiway tree with n keys has exactly $n+1$ empty subtrees, so $n + 1 \ge 2a^{h-1}$, giving $h \le 1 + \log_a \frac{n+1}{2}$. Similarly $n + 1 \le b^h$ gives the lower bound. □

Insert and delete work exactly as for 2-3 trees (split on overflow, transfer or merge on underflow), touching $\mathcal{O}(h)$ nodes.

11.3.4 B-Trees

Definition 11.8. A *B-tree of order m* is an $(\lceil m/2 \rceil, m)$-tree. In the EMM we choose $m \in \Theta(B)$ so that one node fills one block.

Operation	Block transfers	Internal work
`search`	$\Theta(\log_B n)$	$\mathcal{O}(\log B)$ per node (binary search)
`insert`	$\Theta(\log_B n)$	$\mathcal{O}(B)$ per node
`delete`	$\Theta(\log_B n)$	$\mathcal{O}(B)$ per node

Since $\log_B n = \frac{\log n}{\log B}$, a B-tree with $B = 1000$ and $n = 10^9$ has height about 3, and the root (and often level 1) can be kept in internal memory.

Variants.

- **B$^+$-trees**: all KVPs stored in leaves; internal nodes hold only keys for routing, so they have higher fanout; leaves are linked for fast range queries. Used by databases and file systems.
- **Pre-emptive splitting/merging**: split full nodes on the way down during insert, so a single top-down pass suffices.
- **Red-black trees** are a binary encoding of 2-3-4 trees: a 2-3-4 node becomes a black node with 0, 1 or 2 red children.

11.4 Extendible Hashing

Hashing in external memory: we want $\mathcal{O}(1)$ block transfers per operation, and to grow without rehashing everything.

Structure.

- Each key's hash $h(k)$ is a long bit string (say L bits).
- A *directory* (kept in internal memory) is an array of size 2^d, where d is the *global depth*. Entry i points to the block for keys whose hash starts with the d-bit string i.
- Each block (on disk) stores up to $S \approx B$ keys and has a *local depth* $k \leq d$: all keys in the block agree on their first k bits. A block with local depth k is pointed to by 2^{d-k} directory entries.

Search. Read the first d bits of $h(k)$, look up the directory (free), load one block: 1 transfer.

Insert. Search for the block P. If P has room, add k. Otherwise *split* P:

1. If P's local depth $k_P < d$: create a new block P', set both local depths to $k_P + 1$, redistribute the keys of P by bit $k_P + 1$, and redirect half of the directory entries that pointed to P to P'.
2. If $k_P = d$: first *double the directory* ($d \leftarrow d + 1$, each old entry is copied into two), then split as above.

Repeat if all keys went to one side.

```
extendible-insert(k):
  P <- block for the first d bits of h(k)
  while P is full do
    if local-depth(P) = d then double the directory; d <- d + 1
    split P into P and P' by bit local-depth(P) + 1
    update directory pointers
    P <- block for the first d bits of h(k)
  insert k into P
```

Delete. Remove k from its block. Optionally merge a block with its "buddy" (the block differing only in the last of the k bits) if their combined contents fit, and halve the directory when all local depths are less than d.

Example 11.9. $S = 2$ keys per block, $d = 1$: block A (prefix 0, $k_A = 1$) holds keys with hashes `0010`, `0111`; block B (prefix 1) holds `1000`. Insert `0101`: A is full and $k_A = d$, so double the directory to $d = 2$ (entries 00, 01, 10, 11; both 10 and 11 point to B, whose local depth stays 1). Split A by the second bit: $A = \{$`0010`$\}$ (prefix 00), $A' = \{$`0111`, `0101`$\}$ (prefix 01), both with local depth 2.

Analysis. Search and delete: 1 block transfer. Insert: usually 1-2, plus occasionally a directory doubling, which is done in internal memory (no block transfers for the data). No global rehash ever moves all the data; a split touches only one block. Space: blocks may be partly empty (about 69% full on average for random hashes); the directory can blow up if many hashes share long prefixes.

11.5 Summary

- In the EMM we count block transfers of size B, with internal memory M.
- External MergeSort: initial runs of size M, then (M/B)-way merges: $\Theta(\frac{n}{B} \log_{M/B} \frac{n}{M})$ transfers.
- 2-3 trees, (a, b)-trees and B-trees keep all leaves at the same depth using split/transfer/merge; a B-tree of order $\Theta(B)$ uses $\Theta(\log_B n)$ transfers per operation.
- Extendible hashing: a directory on the first d hash bits, blocks with local depths; $\mathcal{O}(1)$ transfers per operation and local splits instead of global rehashing.